Traceable Bottom-Up Secret Sharing and Law & Order on Community Social Key Recovery

Rittwik Hajra[1], Subha Kar[1], Pratyay Mukherjee[1,2], and Soumit Pal[1]

[1] Indian Statistical Institute, Kolkata, India
{rittwik1729, subhakar9732, soumitpal378}@gmail.com
[2] Hashgraph
pratyay85@gmail.com

Abstract. A recent work by Kate et al. [EPRINT 2025] proposes a community-based social recovery scheme (SKR), where key-owners can use a subset of other community members as guardians, and in exchange, they play guardians to support other participants' key recovery. Their construction relies on a new concept called bottom-up secret sharing (BUSS). However, they do not consider a crucial feature, called traceability, which ensures that if more than a threshold number of the guardians collude, at least some colluders' identities can be traced – thereby deterring participants from colluding. In this paper, we incorporate traceability into the community social key recovery as an important feature.
We first introduce the notion of traceable BUSS, which allows tracing colluders by accessing a reconstruction box. Then, extending the work of Boneh et al. [CRYPTO 2024], we propose the first traceable BUSS construction. Finally, we show how to generically use a traceable BUSS scheme to construct a traceable SKR in the aforementioned community setting. Overall, this is the first scheme combining decentralized key management with traceability, marrying BUSS's scalability with the deterrence of traceable secret sharing.

Keywords: Bottom-Up Secret Sharing, Tracing, Social Key Recovery

1 Introduction

Secure key management in decentralized environments – such as cryptocurrency wallets and distributed communities – has become a critical challenge. Unlike passwords, cryptographic secret keys cannot be reset or recovered if lost. The stakes are enormous: it is estimated that over USD 140 billion in Bitcoin alone is permanently inaccessible due to lost keys [40]. To mitigate the risk of irrecoverable loss, users are encouraged to back up their private keys. Traditional approaches include writing down mnemonic phrases or splitting the key via threshold secret sharing (e.g., Shamir's scheme) and distributing shares to different devices or trusted individuals. However, these methods are prone to failure: physical backup can be destroyed or forgotten, and splitting keys among devices or custodians introduces new security vulnerabilities. In scenarios such as the death or incapacitation of a key owner, assets can still be lost if no reliable recovery mechanism is in place.

Bottom-Up Secret Sharing and Social Key Recovery Recently Kate et al. [31] proposed a social key recovery (SKR) mechanism to address the issue. In an SKR scheme, a user (the key owner) entrusts a set of friends or community members – referred to as *guardians* – with the ability to collectively recover her secret key if needed. Typically, the secret key is split and distributed such that any $(t+1)$ out of n guardians can reconstruct the key, while any t or fewer learn nothing. This approach leverages social trust to improve reliability: even if the user loses access to her key, any qualified subset of guardians can help restore it, preventing permanent loss. Social recovery, as advocated by Buterin [16], offers significant usability benefits without relying on centralized custodians.

Central to SKR, Kate et al. [31] proposed a novel secret sharing scheme called Bottom-Up Secret Sharing (BUSS) scheme. In a BUSS scheme, there is no centralized dealer distributing shares; instead, each guardian generates its share independently, using only its own secret information and the identity of the key-owner. Intuitively, each guardian P_j computes a share σ_j as a deterministic function of (i) P_j's personal secret key and (ii) the key-owner's identity or public key. All n guardians' shares are then implicitly associated with a secret s (the key-owner's secret) by some public binding information. In the work of Kate et al. [31], the key-owner collects the guardians' contributions to interpolate a polynomial that hides s, and publishes a small public string (e.g. evaluation points of that polynomial). Later, any $t+1$ guardians can reconstruct s by recomputing their shares locally (using their own secrets) and combining them with the public string. The BUSS paradigm eliminates the need for guardians to *store any extra data beyond their own keys* - a stark contrast to naive use of Shamir's scheme [39], which would require each guardian to hold many shares for different friends and incur linear storage blowup. In a BUSS scheme, each guardian does not store any extra share for the user; instead, shares are derived on the fly from each guardian's own long-term secret.

Traceability and Non-Imputability in Secret Sharing Goyal, Song, and Srinivasan [28] recently introduced *traceable secret sharing* in the context of classical secret sharing: in a traceable scheme, any leaked share information can be traced back to the specific parties who leaked it. In a t-out-of-n threshold secret sharing scheme, suppose $f < t$ servers collude and sell their shares, possibly in an obfuscated way such that they can not be trivially traced back to the owners. In a traceable secret sharing scheme, it should be possible to trace at least one of the corrupted servers given the leaked information. Further, the tracer should be able to produce a proof that implicates the corrupted servers.

Goyal, Song and Srinivasan [28] gave the first definition and construction of a traceable secret sharing scheme. Their construction, however, is not practical as the size of the secret shares is quadratic in the size of the secret.

Boneh, Partap and Rotem [9] propose a new definition of (n, t) threshold traceable secret sharing, which allows them to give the first practical constructions from Shamir's secret sharing and Blakley's secret sharing. In their definition, the Tracer is given black-box access to a "*reconstruction box*" that has

$f < t$ shares hardcoded in it. The Tracer on input the remaining $t - f$ additional shares $\mathcal{R}$ outputs the secret that can be reconstructed from the t shares it now holds. To make the definition, they have introduced two new keys, which are given to the parties along with their shares: these are *trace key* and *verification key*. The scheme is called *traceable* if, given the tracing key, the tracer can find all f parties that own one of the shares hard-coded in $\mathcal{R}$ and produce a proof that implicates these parties. The scheme is called *non-imputable* if the tracer cannot falsely accuse a party by forging a proof of their corruptness. The authors present two schemes that satisfy traceability and non-imputability, one based on Shamir's secret sharing and one based on Blakley's [6] secret sharing scheme. The schemes are practical in the sense that the share size is only twice as large as the size of the secret.

Traceable Solution of BUSS and SKR Prior traceable secret sharing schemes were designed for traditional top-down secret sharing. The first construction by Goyal et al. [28] achieved traceability and non-imputability but at the cost of large shares (quadratic in the secret size) and complex tracing procedures based on Goldreich–Levin hardcore predicates. Subsequent works (e.g., Boneh et al.[9]) improved efficiency, but those solutions still assume a dealer distributing shares and are not directly applicable to the decentralized BUSS context. Boneh, Partap and Rotem [9] suggested the following "*An interesting open question is to devise tracing procedures for other existing secret-sharing schemes.*". No solution currently exists for introducing traceability into bottom-up, guardian-driven secret sharing – and hence into social key recovery – in a way that preserves its practical benefits, i.e., minimal guardian burden and simple, one-round operation. Thus, the following question is natural to ask:

Is it possible to build traceable bottom-up secret sharing schemes and extend the notion of traceability in social key recovery?

Our work fills this gap by presenting the first *Traceable Bottom-Up Secret Sharing* scheme and applying it to build traceable *Social Key Recovery* systems.

1.1 Our Contribution

We present the first traceable bottom-up secret sharing schemes, building on the BUSS framework defined in [31]. Our schemes are practical as they only increase the size of the secret by a factor of 2. We summarize our key contributions as follows:

1. We introduce the concept of traceability in bottom-up secret sharing and provide a concrete Traceable Bottom-Up Secret Sharing (TBUSS) scheme that satisfies the private traceability notion of Boneh, Partap, and Rotem [9]. Notably, guardians still do not store anything beyond their own keys, and share sizes remain compact.

2. Next, we propose a Non-Imputable Traceable BUSS construction, called NITBUSS.
3. Finally, we integrate our TBUSS and NITBUSS schemes with SKR to come up with Traceable and Non-Imputable SKR, respectively. We call them TSKR ("*Traceable* SKR") and NISKR ("*Non-Imputable* SKR"). The TSKR protocol preserves the "bottom-up" nature (guardians only use their own keys and public info) while adding traceability. We also outline an upgraded variant with non-imputability, obtained by using NITBUSS in place of TBUSS in the back-up and recovery process. This gives us NISKR scheme that offers the strongest level of assurance: even in a community social recovery setting, any collusion of up to $f < t + 1$ guardians will be caught with evidence, and no honest guardian can be framed. Together, these contributions lay a new foundation for traceable social key recovery, marrying the scalability of BUSS with the deterrence of traceable secret sharing.

1.2 Technical Overview

We refer the reader to go through the work of [31] and [9] for the notions of BUSS, SKR, and Traceability is Shamir's Secret Sharing, respectively. Our framework builds in layers, each adding a new security property to a standard BUSS scheme. We first show how to make BUSS traceable (TBUSS) using random evaluation points, then strengthen it with error tolerance via Reed–Solomon list decoding. Next, we hide the randomness under one-way functions to achieve non-imputability (NITBUSS). If we incorporate TBUSS in the Share and Reconstruction algorithms of SKR, we yield a *Traceable Social key Recovery* (TSKR) primitive that enforces accountability without non-imputability. Finally, we observe that simply swapping in NITBUSS in the Share/Reconstruction algorithms yields a traceable social key recovery (TSKR) primitive that achieves non mpairability (NISKR).

Traceable BUSS We modify the construction of BUSS and propose a TBUSS construction; this modification is due to assigning random identities to the guardians. Fixed identities will lead to an inefficient tracing procedure; hence, this modification is required. We declare these random identities as their trace keys and verification keys. Suppose $f < t + 1$ guardians are corrupt and hard-coded their shares to the reconstruction box $\mathcal{R}$. If it is a good reconstruction box, then upon receiving the trace key tk along with $t + 1 - f$ extra shares, the box should output the correct secret. It is easy to see if $f = t$ and $\mathcal{R}$ are good; in that case, simple polynomial interpolation will work. So, the shares of the form $(x_i, \sigma_i)_{i=1}^{t}$ are hard-coded in the reconstruction box $\mathcal{R}$ and upon probing $\mathcal{R}$ with two additional shares (x_{t+1}, σ_{t+1}) and $(x_{t+1}, \sigma_{t+1} + 1)$ along with the public value φ, we yield a polynomial $h(X)$ from the difference of two outputs of the reconstruction box $\mathcal{R}$ on input two secrets, respectively. The public value will contribute a factor of $h_\varphi(X)$ in $h(X)$, then by finding the roots of $g(X) := h(X) \cdot (h_\varphi(X))^{-1}$ we obtain all the corrupt guardians' identities $x_1, \ldots, x_t$ as we have assumed.

For imperfect reconstruction boxes, the analysis becomes more complex. Malicious guardians may try to evade tracing by submitting incorrect or tampered shares during an unauthorized reconstruction in the hope of "confusing" the tracing mechanism. To counter this, our TBUSS construction incorporates an error-tolerance layer using Reed–Solomon list decoding. Reed–Solomon codes are a natural fit since our secret sharing is polynomial-based (a form of Reed–Solomon code). We design the tracing algorithm to treat a suspect reconstruction as a codeword and to perform list decoding to recover the candidate sets of shares that could produce that result. Even if some colluders provide erroneous values or noise, the list-decoding procedure can still pinpoint a small list of possible guardian identities, from which the true culprits are extracted. In essence, we can tolerate a bounded number of "errors" (e.g., false shares contributed to misleading tracing) and still successfully trace all the actual participants in the reconstruction. We employ the Guruswami-Sudan [29] algorithm to obtain a list of candidate polynomials G, which is polynomial in size. We perform a search on this list by factoring every polynomial to find the first polynomial whose f roots are basically corrupt guardians' identities $x_1, x_2, \ldots, x_f$.

Adding Non-Imputability A threshold secret sharing scheme is called non-imputable if the tracer cannot falsely accuse a party by forging a proof of their corruptness. To achieve non-imputability, we hide the tracing key tk using one-way functions. In the NITBUSS scheme, the random evaluation points, i.e., the trace key components, are not published in the clear. Let $\mathcal{F} : \mathbb{F} \to \mathbb{F}$ be a one-way function and instead of $(x_1, \ldots, x_N)$ we call $(u_1, \ldots, u_N)$ to be the trace key tk, where $u_i = \mathcal{F}(x_i)$ for all $i \in [N]$. The key owner, without the guardian's secret, cannot forge a consistent share for that guardian. Thus, if the tracer claims that guardian P_i's share was used, it must be that only P_i could have generated that share. Thus, no tracer or party can falsely implicate P_i since any fake evidence would fail to align with the one-way function constraint and would be rejected by the verification algorithm. We formally prove that if the one-way function $\mathcal{F}$ is hard to invert, the probability of a false accusation is negligible. Thus, non-imputability of NITBUSS essentially boils down to the hardness of inverting $\mathcal{F}$.

Traceable and Non-Imputable SKR We integrate the TBUSS and NITBUSS primitives into a full Social Key Recovery application. The integration is seamless: we replace the backup phase of SKR with our TBUSS Share algorithm, and we replace the recovery phase of SKR with our TBUSS Reconstruction algorithm. The resulting Traceable SKR (TSKR) and Non-Imputable SKR (NISKR) protocols preserve the workflow of the original SKR – for example, a user still only needs to perform one round of communication with guardians to back up the key, and can run the recovery algorithm whenever he wishes. The TSKR protocol preserves the "bottom-up" nature (guardians only use their own keys and public info) while adding traceability. The traceability of TSKR follows from the traceability of TBUSS. There are at most N invocations of TBUSS in a TSKR,

for each party P_i for $i \in [N]$, some $f < t+1$ corrupt guardians have hard-coded their shares in a reconstruction box $\mathcal{R}_i$. Thus, we have at most N reconstruction boxes $\mathcal{R}_1, \ldots, \mathcal{R}_N$. The tracer will have access to all these reconstruction boxes and perform the tracing mechanism for every party P_i individually to come up with the corrupt guardians for party P_i for every $i \in [N]$. Note that we have just one trace key which the parties in SKR can generate themselves. So a party P_i participating in SKR only needs to store their secret sk_i, and the shares and trace keys can be computed from this secret via a hash function. By swapping in the $\mathsf{NITBUSS}$ variant, we obtain a Non-Imputable Social Key Recovery (NISKR) system that further guarantees no false evidence can be manufactured against honest guardians. In summary, our technical innovations enable *provably traceable and non-imputable social key recovery*, aligning key management with real-world notions of "*law and order*".

1.3 Related Work

Social Recovery has been an important theoretical as well as practical problem. Smart Contract Based Social Recovery [16,38], Coinbase WaaS: Key-recovery using MPC [34], Account Recovery for a Privacy-Preserving Web Service [35], off-chain backup using 2FA [1], to name a few. For a comprehensive study, we refer to the SoK [17]. Bellare, Dai, and Rogaway's work on Adept Secret Sharing [4] is a key contribution that connects Social Key Recovery with traditional Secret Sharing. Alongside the usual privacy guarantee, Adept Secret Sharing introduces two extra features: authenticity and error correction. The authenticity feature lets anyone confirm that the recovered secret is indeed the right one. To make this possible, a commitment to the secret is first stored in a trustworthy public place, and later compared against the reconstructed secret.

Chor, Fiat, and Naor [19] first introduced traitor tracing for broadcast encryption, which lets a tracer identify the source of any leaked decryption keys. Many traitor tracing methods have been developed since then [12,33,37], [7], [23], [32], [20], [11], [8], [24], [13], [27], [18], [42], [41], but these differ from the approach taken by Boneh, Partap, and Rotem [9] and in our work. For a comprehensive review of these techniques, refer [9].

After the inception of traceable secret sharing by Goyal, Song, and Srinivasan [28], there have been many developments towards this direction. One important milestone will be the Boneh et al.'s work [9], which simplifies the syntax and definitions and presents two important traceable secret sharing schemes: Shamir and Blakely. Recently, Hoffman [30] presented a traceable version of a Chienese Remainder Based secret sharing scheme called Mignotte secret sharing scheme [36]. The work of Boneh, Partap and Rotem [9] gives rise to several important work: Traceable Verifiable Secret Sharing (VSS) [26,2], Traceable verifiable Random Function (VRF) [10], Traceable General Access Structure based secret sharing schemes [22], Traceable Threshold Encryption [5,15,9], Secret Sharing with Snitching [21,14], to name a few.

1.4 Organization of the Paper

In Section 2 we develop notations and the definitions and security games, used throughout the paper. In Section 3, we write the Traceable variant of BUSS and the full-fledged tracing algorithm along with the security theorem. Section C demonstrates a non-imputable variant of BUSS. In the section 5 we give a traceable version of SKR. Then we compare our tracing mechanisms with the existing literature in the section 6. Finally, we conclude in section 7.

2 Preliminaries

2.1 Notation

We use $\mathbb{N}$ to denote the set of positive integers. For a natural number $n \in \mathbb{N}$ we denote $[n] := \{1, 2, \ldots, n\}$. Let $\mathbb{F}$ denote a finite field of a certain order unless specified by indices. A ordered tuple $(x_1, x_2, \ldots, x_n)$ is denoted by vector notation $\boldsymbol{x}_{[n]}$. Similarly for any subset $S \in [n]$, $\boldsymbol{x}_S$ and $(x_i)_{i \in S}$ are defined accordingly. For any finite set $\mathcal{D}$, we let $x \xleftarrow{\$} \mathcal{D}$ denote picking an element of $\mathcal{D}$ uniformly at random and assigning it to x. We also use the concepts of one-way function in Section C, for the definition and details reader is referred to [25].

2.2 Syntax of Bottom-Up Secret Sharing (BUSS) and Social Key Recovery (SKR)

BUSS is introduced in the work of Kate et al. [31]. Consider a set of parties $P_1, P_2, \ldots, P_N$, where each party P_i has a unique identity $i \in [N]$, which enables each party to generate its share independently of both the other parties' shares and the underlying secret. In BUSS, each share is deterministically derived from the guardian's own secret key and the identity of the key-owner, ensuring that guardians are not required to store or manage any information beyond their own secret keys. The key-owner interpolates these shares into a secret-sharing polynomial and publishes a small set of public evaluation points. During recovery, any $t+1$ guardians can locally recompute their respective shares, and together with the public values, reconstruct the original secret. Note that in the following definition, N denotes the universe size, i.e., the total number of parties in a network, but for every party $i \in [N]$, the number of guardians is fixed, i.e., $n-1$.

Definition 1 (Bottom-Up Secret Sharing (BUSS) [31]). *For $n, t, N \in \mathbb{N}$ such that $t+1 \leq n-1 < N$, a bottom-up secret sharing scheme for a $(t+1)$-out of -$(n-1)$ threshold access structure over the set $[N]$ consists of the following two algorithms:*

- $\mathsf{Share}(1^\lambda, s, \boldsymbol{\sigma}_B, B)$: *On input s, $(n-1)$ shares $\boldsymbol{\sigma}_B$, and the corresponding set of indices $B \subseteq [N] \setminus \{i\}$ such that $|B| = n-1$. Then* Share *outputs a public value φ.*

– $\mathsf{Recon}(\varphi, \boldsymbol{\sigma}_R, R)$: *On input* φ, $(t+1)$ *shares* $\boldsymbol{\sigma}_R$, *and the corresponding set of indices* $R \subseteq [N]$ *such that* $|R| = t+1$. *Then* Recon *outputs a secret* s *or* $\perp$.

Correctness For any secret s, any sets R, B such that $R \subseteq B \subset [N]$, with $|R| = t+1, |B| = n-1$ and any $\boldsymbol{\sigma}_B$:

$$\Pr[s \leftarrow \mathsf{Recon}(\varphi, \boldsymbol{\sigma}_R, R) \mid \varphi \leftarrow \mathsf{Share}(1^\lambda, s, \boldsymbol{\sigma}_B, B)] = 1$$

Perfect Adaptive Simulation Security A scheme $\mathsf{BUSS} = (\mathsf{Share}, \mathsf{Recon})$ for a $(t+1)$-out-of-$(n-1)$ access structure is **perfect adaptive simulation secure** if for every unbounded adversary $\mathcal{A}$, there exist simulators $\mathsf{SimShare}, \mathsf{SimComb}$ such that

$$\forall \mathcal{A}, \quad \Pr[\mathsf{RealSh}_{\mathcal{A}}(\lambda) = 1] = \Pr[\mathsf{IdealSh}_{\mathcal{A}}(\lambda) = 1]$$

– $\underline{\mathsf{RealSh}_{\mathcal{A}}}$
  - Receive $(s, \boldsymbol{\sigma}_C, C, B)$ from $\mathcal{A}$, such that $C \subseteq B$, $|C| \leq t$, and $|B| = (n-1)$.
  - Sample $\boldsymbol{\sigma}_{B \setminus C}$ uniformly at random.
  - Run $\varphi \leftarrow \mathsf{Share}(s, \boldsymbol{\sigma}_B, B)$ and give φ to $\mathcal{A}$.
  - When $\mathcal{A}$ queries (Share, C') for $C' \subseteq B \setminus C$: Update $C := C \cup C'$. If $|C| > t$, abort, else, give $\boldsymbol{\sigma}_{C'}$ to $\mathcal{A}$.
  - Finally, give $\boldsymbol{\sigma}_{B \setminus C}$ to $\mathcal{A}$.
  - Output whatever $\mathcal{A}$ returns.
– $\underline{\mathsf{IdealSh}_{\mathcal{A}}}$
  - Receive $(s, \boldsymbol{\sigma}_C, C, B)$ from $\mathcal{A}$, such that $C \subseteq B$, $|C| \leq t$, and $|B| = (n-1)$.
  - Run $(\varphi, \boldsymbol{\tau}) \leftarrow \mathsf{SimShare}(C)$ and give φ to $\mathcal{A}$.
  - When $\mathcal{A}$ queries (Share, C') for $C' \subseteq B \setminus C$:Update $C := C \cup C'$ and set $\boldsymbol{\sigma}_{C'} := \boldsymbol{\tau}_{C'}$. If $|C| > t$, abort, Else, give $\boldsymbol{\sigma}_{C'}$ to $\mathcal{A}$.
  - Compute $\boldsymbol{\sigma}_{B \setminus C} \leftarrow \mathsf{SimComb}(s, B, C, \boldsymbol{\sigma}_C, \varphi)$ and give it to $\mathcal{A}$.
  - Output whatever $\mathcal{A}$ returns.

In the following, we revisit the SKR definition from [31]. In a decentralized community, each individual held a private key—generated securely through a process called KeyGen—that unlocked access to their digital assets. But what if someone lost their key? Rather than relying on fragile memories or risky storage, the community adopted a social recovery scheme. In this system, no one needed to remember anything beyond their own secret key. When a person backed up their key using the protocol Π_{Back}, the only result was a piece of public information—no hidden shards, no sensitive data passed around. The guardians, friends, and peers in the network didn't bear the burden of remembering secret pieces of someone else's life. It was simple and elegant. Though built for a common key generation process, the system was flexible enough to evolve, even if each person used a different method to create their key. Trust stayed local, responsibility stayed minimal, and recovery remained possible—together

Definition 2 (Social Key Recovery Scheme (SKR)[31]). *Consider a set of N parties $P_1, \ldots, P_N$, and let $\mathcal{U}$ be an access structure defined by pairs of sets (B, R), where $B \subseteq [N]$ and $R \subseteq B$. Without loss of generality, we assume that each party P_i is associated with a public identity-i. Let KeyGen denote a key generation algorithm that outputs a key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$. A social key recovery scheme Π_{SKR} for the parties $P_1, \ldots, P_N$, the algorithm KeyGen, and the access structure $\mathcal{U}$ consists of three protocols $(\Pi_{\mathsf{Init}}, \Pi_{\mathsf{Back}}, \Pi_{\mathsf{Rec}})$ executed sequentially by subsets of the parties as follows: all parties first run Π_{Init}; subsequently, any party may initiate a single execution of Π_{Back} with a chosen subset $B \subseteq [N]$; and finally, once P_i has completed Π_{Back}, it may invoke Π_{Rec} any number of times. The syntax of the protocols is defined as follows:*

- Π_{Init}*: In this protocol, a party P_i locally generates a key pair $(\mathsf{sk}_i, \mathsf{pk}_i)$ either by computing $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda)$ or using another algorithm specified by the protocol. Each party P_i publishes pk_i. An execution is denoted by:*

$$(\mathbf{pk}_{[N]}, \mathbf{sk}_{[N]}) \leftarrow \Pi_{\mathsf{Init}}(1^\lambda, 1^N)$$

- Π_{Back}*: In this protocol, a key owner P_i who wishes to back up her secret key sk_i interacts with a set of guardians $\{P_j\}_{j \in B}$. Each guardian P_j uses secret key sk_j. The protocol concludes with a public backup string pub_i. We denote such an execution by:*

$$\mathsf{pub}_i \leftarrow \Pi_{\mathsf{Back}}(i, B, \mathbf{pk}_B, \mathbf{sk}_{B \cup \{i\}})$$

- Π_{Rec}*: If P_i wishes to recover sk_i using a recovery set R, she runs this protocol (without any secret input) with a set of guardians $\{P_j\}_{j \in R}$, each of which uses their secret key sk_j. In addition, pub_i may be used by all parties. At the end of this protocol, the key-owner may receive a private output sk_i (or $\perp$ if unsuccessful). One such execution is denoted as:*

$$\mathsf{sk}_i / \perp \leftarrow \Pi_{\mathsf{Rec}}(i, R, \mathbf{pk}_R, \mathsf{pub}_i, \mathbf{sk}_R)$$

Correctness. *For correctness we require that for any sufficiently large λ, any $i \in [N]$, any $(B, R) \in \mathcal{U}$:*

$$\Pr\left[\mathsf{sk}_i \leftarrow \Pi_{\mathsf{Rec}}(i, R, \mathbf{pk}_R, \mathsf{pub}_i, \mathbf{sk}_R) \;\middle|\; \begin{array}{c} (\mathbf{pk}_{[N]}, \mathbf{sk}_{[N]}) \leftarrow \Pi_{\mathsf{Init}}(1^\lambda, 1^N); \\ \mathsf{pub}_i \leftarrow \Pi_{\mathsf{Back}}(i, B, \mathbf{pk}_B, \mathbf{sk}_{B \cup \{i\}}) \end{array} \right] = 1$$

2.3 Syntax of Traceable Bottom-Up Secret Sharing Scheme (TBUSS)

TBUSS combines the scalability of decentralized social key recovery with the accountability of traceable secret sharing. In BUSS, each user derives backup shares for others using only their own secret key, avoiding additional storage and enabling efficient $t+1$-out-of-$n-1$ recovery with the help of publicly posted values. By integrating traceability, where leaked shares or reconstruction mechanisms

can be traced back to the responsible parties. We mostly follow the definitions in [9]. However, we will tune their notions accordingly to cater BUSS scheme, as BUSS is not perfectly private. Although it has been shown that BUSS is Perfect Adaptive Simulation Secure [31]. But for our purpose of Tracing, we will not need those security notions.

Definition 3 (TBUSS). *For parameters $n, t, N \in \mathbb{N}$ where $t+1 \leq n-1 < N$, a* TBUSS *scheme for a $(t+1)$-out-of-$(n-1)$ threshold access structure over $[N]$ is a 4-tuple of algorithms* $\mathsf{TBUSS} = (\mathsf{Share}, \mathsf{Recon}, \mathsf{Trace}, \mathsf{Verify})$*:*

- $\mathsf{Share}(1^\lambda, s, \boldsymbol{\sigma}_B, B) \to (\varphi, \mathsf{tk}, \mathsf{vk})$*: On input s, $(n-1)$ shares $\boldsymbol{\sigma}_B$, and the corresponding set of indices $B \subseteq [N] \setminus \{i\}$ such that $|B| = n-1$. Then outputs a public value φ and tracing key* tk*, and verification key* vk*.*
- $\mathsf{Recon}(\varphi, \boldsymbol{\sigma}_R, R) \to s$ *or* $\perp$*: On input φ, $(t+1)$ shares $\boldsymbol{\sigma}_R$, and the corresponding set of indices $R \subseteq [N]$ such that $|R| = t+1$. Then* Recon *outputs a secret s or $\perp$.*
- $\mathsf{Trace}^{\mathcal{R}}(\mathsf{tk}) \to (I, \pi)$*: It is a randomized algorithm that takes the tracing key* tk *and oracle access to a reconstruction box $\mathcal{R}$. It outputs a subset $I \subseteq [n-1]$ of corrupted parties and a proof π.*
- $\mathsf{Verify}(\mathsf{vk}, I, \pi) \to \{0,1\}$ *is a deterministic algorithm that takes the verification key* vk*, the set I, and proof π, and returns* 1 *if the proof is valid and* 0 *otherwise.*

Correctness. The correctness requirement for a TBUSS is same as BUSS, any set of $t+1$ shares with the public value φ must be sufficient to correctly reconstruct the original secret. In other words, the reconstruction algorithm should output the correct secret whenever it is given any $t+1$-tuple of valid shares produced by the sharing algorithm.

Definition 4 (ϵ-Correctness). *Let* $\mathsf{TBUSS} = (\mathsf{Share}, \mathsf{Recon}, \mathsf{Trace}, \mathsf{Verify})$ *be a Traceable Bottom-Up Secret Sharing scheme and let $\epsilon \in [0,1]$ be a function of security parameter λ. We say that* TBUSS *is ϵ-correct if for every $\lambda \in \mathbb{N}$,every secret s every $0 < t+1 \leq n$ and every pairs of sets (B, R), where $B \subseteq [N]$ and $R \subseteq B$ and $|B| = n-1, |R| = t+1$, it holds that*

$$\Pr[s' = s] \geq 1 - \epsilon.$$

where $(\varphi, \mathsf{tk}, \mathsf{vk}) \leftarrow \mathsf{Share}(1^\lambda, s, \boldsymbol{\sigma}_B, B)$ *and* $s' := \mathsf{Recon}(\varphi, \boldsymbol{\sigma}_R, R)$,

In the context of traceable secret-sharing schemes, it is natural to strengthen the standard secrecy requirement by demanding that the tracing algorithm itself does not compromise the confidentiality of the secret. Specifically, the tracing key tk, which is designed to facilitate the identification of corrupted parties, should not reveal any additional information about the shared secret. Even if an adversary gains access to both the tracing key and fewer than $(t+1)$ shares, they should still be unable to distinguish between different possible secrets. This intuition is formally captured in the definition of tracer secrecy provided below.

We define a Traceability game in Figure 1. The following definition will give us the traceability advantage.

$\mathsf{Game.Trace}_{\mathcal{A},\mathsf{TBUSS},\varepsilon}(\lambda)$

1. $\mathcal{A}(1^\lambda, n-1, t+1, f)$ outputs $(\mathcal{I}, \mathsf{state})$, where $\mathcal{I} \subset [n-1]$ and $|\mathcal{I}| = f(< t+1)$ is the set of parties to corrupt.
2. $s \xleftarrow{\$} \mathcal{S}$ and $\boldsymbol{\sigma}_B \xleftarrow{\$} |\mathbb{F}|^{|B|}$.
3. $(\varphi, \mathsf{tk}, \mathsf{vk}) \leftarrow \mathsf{Share}(s, \boldsymbol{\sigma}_B, B)$
4. On input all shares of parties in $\mathcal{I}$, the adversary $\mathcal{A}(\mathsf{state}, \sigma_{i_1}, \ldots, \sigma_{i_f})$ outputs a reconstruction box $\mathcal{R}$.
5. $\mathsf{Trace}^{\mathcal{R}}(\mathsf{tk})$ outputs $(\mathcal{I}', \pi)$.
6. $\mathcal{A}$ wins if: $\mathcal{R}$ reconstructs the secret from consistent inputs with probability at least ε, i.e., $\mathcal{R}$ is good and Either $\mathcal{I} \neq \mathcal{I}'$ or $\mathsf{Verify}(\mathsf{vk}, \mathcal{I}', \pi) = 0$.

Fig. 1: The tracing game for traceable bottom-up secret sharing BUSS

Definition 5 (Traceability). *Let* $\mathsf{TBUSS} = (\mathsf{Share}, \mathsf{Rec}, \mathsf{Trace}, \mathsf{Verify})$ *be a Traceable Bottom-Up Secret Sharing Scheme. Let* $\varepsilon = \varepsilon(\lambda)$ *be a function of the security parameter. We say that* TBUSS *satisfies traceability if for every PPT adversary* $\mathcal{A}$*, the following function is negligible in* λ *(See Figure 1):*

$$\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TBUSS},\varepsilon}(\lambda) := \Pr\left[\mathsf{Game.Trace}_{\mathcal{A},\mathsf{TBUSS},\varepsilon}(\lambda) = 1\right].$$

The above definition states that, for every probabilistic polynomial time adversary, $\mathcal{A}$, the probability that it wins the game $\mathsf{Game.Trace}_{\mathcal{A},\mathsf{TBUSS},\varepsilon}(\lambda)$ defined in Figure 1 is negligible in λ.

In the setting of TBUSS schemes, it is essential to ensure the property of non-imputability, which guarantees that honest participants cannot be falsely accused of leaking information. This security notion asserts that even a malicious tracer (possibly attempting to manipulate the tracing mechanism) should not be able to generate a convincing proof that implicates an honest party who did not participate in any wrongdoing. In other words, the scheme must be robust against forgery of tracing evidence, thereby protecting honest users from being unjustly held accountable.

Definition 6 (Non-Imputability). *A* $\mathsf{TBUSS} = (\mathsf{Share}, \mathsf{Rec}, \mathsf{Trace}, \mathsf{Verify})$ *is said to be satisfied non-imputability if for every PPT adversary* $\mathcal{A}$*, the probability that it wins the game* $\mathsf{GNon\text{-}Imputability}_{\mathcal{A},\mathsf{TBUSS}}(\lambda)$ *is negligible in* λ*:*

$$\mathsf{Adv}^{\mathsf{ni}}_{\mathcal{A},\mathsf{NITBUSS}}(\lambda) := \Pr\left[\mathsf{GNon\text{-}Imputability}_{\mathcal{A},\mathsf{TBUSS}}(\lambda) = 1\right]$$

The above definition states that, for every PPT adversary $\mathcal{A}$ the probability that it wins the game GNon-Imputability is negligible in λ.

3 Traceability in BUSS

In this section, we present a traceable version of the BUSS scheme, and by abuse of notation, we call it TBUSS. We then present a tracing algorithm for TBUSS.

$\mathsf{GNon\text{-}Imputability}_{\mathcal{A},\mathsf{TBUSS}}(\lambda)$

1. $\mathcal{A}(1^\lambda, n, t) \to (i^*, s, B, \mathsf{state})$.
2. $\mathsf{Share}(s, B, \boldsymbol{\sigma}_B) \to (\varphi, \mathsf{tk}, \mathsf{vk})$.
3. On input all shares *except* for the i^*-th one and the keys tk and vk,
   $\mathcal{A}(\mathsf{state}, \boldsymbol{\sigma}_B \setminus \{\sigma_{i*}\}, \mathsf{tk}, \mathsf{vk})$ outputs $(\mathcal{I}^*, \pi)$.
4. $\mathcal{A}$ wins if $i^* \in \mathcal{I}^*$ and $\mathsf{Verify}(\mathsf{vk}, \mathcal{I}^*, \pi) = 1$.

Fig. 2: The non-imputability game for traceable bottom-up secret sharing TBUSS.

3.1 Construction of TBUSS

In BUSS, assigning fixed evaluation points (e.g., $x_i = i$) lets a malicious reconstructor $\mathcal{R}$ identify which party a share came from and reject queries involving corrupted parties. For example, if f parties are corrupted, $\mathcal{R}$ can output $\perp$ unless it receives exactly $t - f$ honest shares, making successful tracing negligible unless queries are exponential in n. To avoid this, each party is assigned a secret, random evaluation point $x_i \xleftarrow{\$} \mathbb{F}$, breaking the link between shares and party identities. Provided the field $\mathbb{F}$ is large enough, the probability of a collision $x_i = x_j$ is negligible, and any resulting correctness error can be further reduced via rejection sampling. This simple change is essential to enable efficient black-box tracing.

In this section, we provide a construction of a TBUSS, for which we borrow the underlying BUSS construction of [31]. To enable efficient tracing, we modify the original BUSS construction by replacing the fixed evaluation point with a randomly chosen one. Specifically, in the Share algorithm of the standard BUSS, each party is identified by an index i and the share is defined as $q(i) = \sigma_i$ for a suitable polynomial f. In contrast, in our traceable variant TBUSS, we assign to each party a random field element $x_i \xleftarrow{\$} \mathbb{F}$ and set $q(x_i) = \sigma_i$, we assume that $x_i \neq x_j$ for all $1 \leq i < j \leq (n-1)$. This randomization facilitates tracing while preserving the original scheme's structure. Let us construct the following TBUSS scheme for a $(t+1)$-out of-$(n-1)$ access structure over a finite field $\mathbb{F}$ in the Figure 3. Correctness and Perfect Adaptive Simulation Security notions in our scheme TBUSS, described in Figure 3, will follow from the work of Kate et al. [31].

3.2 Tracing via polynomial Interpolation

For tracing purposes, we assume that the tracer has access to a reconstruction oracle, denoted $\mathcal{R}$, which allows query-based interaction. Suppose there are $f < t + 1$ corrupted parties whose shares have been hard-coded into $\mathcal{R}$ by the adversary. The tracer can then submit $t - f + 1$ honest shares along with the public value φ to $\mathcal{R}$, which responds with the reconstructed secret s. For simplicity, let us consider the case where t corrupted parties are controlled by the

Share$(1^\lambda, s, \boldsymbol{\sigma}_B, B)$

1. Define a polynomial q over $\mathbb{F}$ of degree $(n-1)$ such that:
   - $q(0) := s$
   - Parse $\boldsymbol{\sigma}_B \to \{(x_j, \sigma_j)\}_{j \in B}$
   - For all $j \in B$ and set $q(x_j) := \sigma_j$
2. Sample $x_{-1}, x_{-2}, \ldots, x_{-(n-t-1)} \xleftarrow{\$} \mathbb{F} \setminus \{x_j\}_{j \in B}$.
3. Compute $\varphi := (\varphi_1, \varphi_2, \ldots, \varphi_{n-t-1})$, where $\varphi_i = (x_{-i}, q(x_{-i}))$.
4. For all $j \in B$, set $\mathsf{tk}_j \leftarrow x_j$ and $\mathsf{vk}_j \leftarrow x_j$, and denote $\mathsf{tk} = (\mathsf{tk}_1, \mathsf{tk}_2, \ldots, \mathsf{tk}_{n-1})$ and $\mathsf{vk} = (\mathsf{vk}_1, \mathsf{vk}_2, \ldots, \mathsf{vk}_{n-1})$.
5. Output $(\varphi, \mathsf{tk}, \mathsf{vk})$

Recon$(\varphi, \boldsymbol{\sigma}_R, R)$

1. Parse φ as $(\varphi_1, \varphi_2, \ldots, \varphi_{n-t-1})$ where each $\varphi_i = (x_{-i}, q(x_{-i}))$.
2. Concatenate these $(n-t-1)$ public points with the $(t+1)$ shares $\{(x_j, \sigma_j)\}_{j \in R}$.
3. Use Lagrange interpolation over these n points to compute the unique polynomial q of degree $(n-1)$
4. Output $s := q(0)$

Fig. 3: The share and reconstruction algorithm of TBUSS.

adversary, i.e., $f = t$. In this setting, the reconstruction oracle $\mathcal{R}$ receives an additional share (beyond those hardcoded) along with the public value φ as input, and returns the reconstructed secret s. This secret corresponds to the value $r(0)$ of a polynomial r, reconstructed using $(n-t-1)$ public evaluations φ, the t hardcoded evaluations of r embedded in $\mathcal{R}$, and the one additional evaluation provided as input. Suppose the shares hardcoded in the reconstruction oracle $\mathcal{R}$ are of the form $(x_i, \sigma_i = q(x_i))_{i=1}^{t}$. Additionally, the public evaluations are given by $(x_j, \sigma_j = q(x_j))_{j=-(n-t-1)}^{-1}$ following a technique of using negative indices stated in [3], and the tracer provides one additional share (x_{t+1}, σ_{t+1}) as input. Given these inputs, the reconstruction oracle $\mathcal{R}$ is expected to output the secret s such that $s = q(0)$, where q is the unique polynomial of degree $(n-1)$ that interpolates all the provided points. In terms of Lagrange Interpolation, we will arrive at the following expression: $s = \sum_{\substack{i=-(n-t-1) \\ i \neq 0}}^{t+1} \left(\prod_{\substack{k=-(n-t-1) \\ k \neq i, 0}}^{t+1} \frac{x_k}{x_k - x_i} \right) \sigma_i$. Otherwise, this is not a good reconstruction box. Upon feeding the reconstruction box with $\mathcal{R}$ with (x_{t+1}, σ_{t+1}) and $(x_{t+1}, \sigma_{t+1} + 1)$ to yield s and s' respectively. Hence, we obtain, The above expression can be re-written as the following: $s = \sum_{\substack{i=-(n-t-1) \\ i \neq 0}}^{t} \left(\prod_{\substack{k=-(n-t-1) \\ k \neq i, 0}}^{t+1} \frac{x_k}{x_k - x_i} \right) \sigma_i + \left(\prod_{\substack{k=-(n-t-1) \\ k \neq 0}}^{t} \frac{x_k}{x_k - x_{t+1}} \right) \sigma_{t+1}$ Now, if we feed the reconstruction box $\mathcal{R}$ with the share $(x_{t+1}, \sigma_{t+1} + 1)$, for the same x_{t+1} and σ_{t+1} as before, we obtain the following expression as before, $s' =$

$\sum_{\substack{i=-(n-t-1)\\ i\neq 0}}^{t} \left(\prod_{\substack{k=-(n-t-1)\\ k\neq i,0}}^{t+1} \frac{x_k}{x_k - x_i}\right)\sigma_i + \left(\prod_{\substack{k=-(n-t-1)\\ k\neq 0}}^{t} \frac{x_k}{x_k - x_{t+1}}\right)(\sigma_{t+1} + 1)$.

Now, subtracting the last two equations, we obtain the following expression, $(s' - s)^{-1} = \prod_{\substack{k=-(n-t-1)\\ k\neq 0}}^{t} \left(\frac{x_k - x_{t+1}}{x_k}\right)$ Now, let us consider the polynomial $h(X) = \prod_{\substack{k=-(n-t-1)\\ k\neq 0}}^{t} \left(\frac{x_k - X}{x_k}\right)$, where the variable is X.

Notice that the polynomial h has as its roots exactly the values x_i of the corrupted parties, together with the public points. Since we only need to identify the corrupted parties, we can ignore the public roots and keep the rest. Observe that the equation of s' represents an evaluation of h at the new point x_{t+1} provided to $\mathcal{R}$. If we repeat this procedure with t additional fresh points x_{t+1}, we obtain $t + 1$ distinct evaluations of h. Because $\deg(h) = t$, these $t + 1$ samples suffice to interpolate h uniquely. Once h is recovered, we factor it to reveal all its roots. Since the tracer's key tk already contains every party's true x_i, each root can be matched back to the corresponding corrupted party.

Although the full degree of h is actually $n - 1$, one can first simplify the expression for $h(X)$ before performing interpolation and factorization. $h(X) = \prod_{k=-(n-t-1)}^{-1} \left(\frac{x_k - X}{x_k}\right) \cdot \prod_{k=1}^{t} \left(\frac{x_k - X}{x_k}\right) = h_\varphi(X) \cdot \prod_{k=1}^{t} \left(\frac{x_k - X}{x_k}\right)$

Note that the polynomial $h_\varphi(X)$ is public as φ is a public value and thus h_φ can be constructed publicly. Hence we arrive at the following polynomial, $g(X) := (s' - s)^{-1} \cdot (h_\varphi(X))^{-1} := \prod_{k=1}^{t} \left(\frac{x_k - X}{x_k}\right)$.

3.3 Tracing Imperfect Reconstruction Boxes

In the previous section, we assumed that the reconstruction oracle $\mathcal{R}$ is good, i.e., it always returns the correct secret. However, this assumption does not hold in general. In reality, the reconstruction box may not always produce the correct output. We refer to such a scenario as an imperfect reconstruction box, where $\mathcal{R}$ returns the correct reconstructed secret only with some non-negligible probability. This imperfection arises because standard polynomial interpolation can fail or yield an incorrect result when some of the provided evaluation points are erroneous. To address this issue, we observe that the task of reconstructing a polynomial of bounded degree from a set of possibly corrupted evaluation points is closely related to the list decoding problem for Reed-Solomon codes [29].

Definition 7 (The Reed-Solomon List Decoding Problem). *Let $\mathbb{F}$ be a finite field, and let $k, M, C \in \mathbb{N}$ such that $C \leq M < |\mathbb{F}|$. Given k, C and M pairs of elements $\{(x_i, \sigma_i)\}_{i=1}^{M} \in \mathbb{F}^2$, output a list G of univariate polynomials of degree at most k that agree with at least C pairs $\{(x_i, \sigma_i)\}$. In particular,*

$$G = \{g \in \mathbb{F}[X] \mid deg(g) \leq k \ \wedge \ |\{j \in [M] \mid g(x_j) = \sigma_j\}| \geq C\}$$

Theorem 1 (Guruswami-Sudan Algorithm [29]). *Let $\mathbb{F}$ be a finite field, and let $k, M, C \in \mathbb{N}$ such that $C \leq M < |\mathbb{F}|$ and $C \geq \sqrt{kM}$. Then, there exists an algorithm $\mathcal{D}_{\mathsf{GS}}$ that solves the list decoding problem as defined in Definition 7 in time polynomial in M, k and $\log |\mathbb{F}|$.*

Since the runtime of the algorithm is polynomial, the list G it outputs will be of polynomial length. We denote $\tau(M, k, \log |\mathbb{F}|)$ to be the polynomial bounding the list length, i.e., $|G| \leq \tau$, where the input to the τ polynomial is understood from the context.

3.4 Tracing Algorithm

Our enhanced tracing procedure starts by gathering the evaluations of the polynomial $g(X)$ as outlined previously. Rather than performing exact interpolation, we apply a list-decoding algorithm to produce several candidate polynomials. We then factor each candidate in turn and verify whether all of its roots appear in the true set of x_i values held by the parties. Almost certainly, just one of these candidates will meet that requirement—its roots reveal the identities of the corrupt parties. Below, we present the formal description of this algorithm, which depends on the Guruswami–Sudan parameters M and C. We'll explain how to choose these values later in the section.

Theorem 2. *Let TBUSS be the Traceable Bottom-Up Secret Sharing scheme defined in Section 3.1. For every adversary $\mathcal{A}$, security parameter $\lambda \in \mathbb{N}$, parameters $M, C \in \mathbb{N}$, and $\epsilon \in \left[\sqrt{\frac{2(M+n+t^2-ft)}{p}}, 1\right]$ with $\epsilon > \sqrt{2C/M}$ and $\sqrt{fM} \leq C \leq M < p$, we have:*

$$\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TTSS},\varepsilon}(\lambda) \leq e^{\frac{-\epsilon^2 M}{2}\cdot(1-1/r)^2} + \frac{f \cdot (n-f-1) \cdot \tau}{p}$$

where $|\mathbb{F}| = p$ (field size), $r = \dfrac{\epsilon^2 M}{2C}$, $n = n(\lambda)$ (size of guardian set $|B| = n-1$), $f = f(\lambda)$ (number of corruptions, $f < t+1$), $\tau = \tau(M, f, \log p)$ (list size from Guruswami-Sudan Algorithm $\mathcal{D}_{\mathsf{GS}}$).

The proof of Theorem 2 is involved and mathematically rich. While proving the theorem 2, we had to take care of some new bad events that did not appear in the proof of Theorem 2 in the work of [9]. The detailed proof with careful analysis of these new bad events is discussed in the full version.

Learning f. If the tracing algorithm does not know the number of corrupted parties f^*, it tests values starting from $f = t-1$ downward until successful. If it guesses $f > f^*$, any identified set of size f necessarily includes an honest party; by Theorem 2, this occurs with probability at most $\frac{f(n-f-1)\tau}{p} \leq \frac{n^2\tau}{p}$. For the correct guess $f = f^*$, the failure probability is at most $e^{-\frac{\epsilon^2 M}{8}} + \frac{n^2\tau}{p}$, which further reduces to $e^{-\lambda} + \frac{n^2\tau}{p}$ since $M \geq 8\lambda/\epsilon^2$. By a union bound, the overall probability that the tracing algorithm identifies the correct corrupted set is at least $1 - (e^{-\lambda} + 2n^3\tau/p)$.

Algorithm: $\mathsf{Trace}^{\mathcal{R}}_{\mathsf{TBUSS}}(\mathsf{tk}, f, 1^{1/\varepsilon})$

1. Parse tk as $(x_1, x_2, \ldots, x_{n-1})$
2. For $\ell = 1$ to M do:
   - For $i = f+1, \ldots, t$: sample $x_{\ell,i}, \sigma_{\ell,i} \xleftarrow{\$} \mathbb{F}$ and set $y_{\ell,i} \leftarrow (x_{\ell,i}, \sigma_{\ell,i})$.
   - Sample $x'_\ell, \sigma'_\ell, \delta_\ell \xleftarrow{\$} \mathbb{F}$. Let $y'_\ell \leftarrow (x'_\ell, \sigma'_\ell)$ and $y''_\ell \leftarrow (x'_\ell, \sigma'_\ell + \delta_\ell)$.
   - Query $\mathcal{R}$ on $(y'_\ell, y_{\ell,f+1}, \ldots, y_{\ell,t})$ and $(y''_\ell, y_{\ell,f+1}, \ldots, y_{\ell,t})$ to obtain s_ℓ and s'_ℓ respectively. This computation involves the public value φ.
   - If $s_\ell = s'_\ell$ or $\delta_\ell = 0$ or $x_{\ell,i} = x'_\ell$ for some $i \in \{f+1, \ldots, t\}$, or if $\ell > 1$, or $x'_\ell = x'_i$ for some $i \in [\ell - 1]$, and $h_\varphi(x'_\ell) = 0$ then terminate and output $\perp$. Otherwise, let $z_\ell \leftarrow \dfrac{\delta_\ell}{(s'_\ell - s_\ell) \cdot h_\varphi(x'_\ell)} \times \left(\prod_{i=f+1}^{t} \dfrac{x_{\ell,i}}{x_{\ell,i} - x'_\ell} \right)$
3. Let $L = \{x'_j, z_j\}_{j \in [M]}$. Run the list decoding algorithm $\mathcal{D}_{\mathsf{GS}}$ on the following: the degree bound f, the integer C and the list L of pairs in $\mathbb{F}^2$. $\mathcal{D}_{\mathsf{GS}}$ returns a list of all candidate polynomials $G = \{g_i\}$ of degree at most f that agree with at least C evaluation points out of L.
4. For each $g_i \in G$ find all roots $w_{i,1}, \ldots, w_{i,f}$ of g_i. If for any $j \in [f]$ there is no k such that $w_{i,j} = x_k$, then remove g_i from the list G. If G is empty, then terminate and output $\perp$.
5. Let g_{j^*} be the first polynomial in G, let $\mathcal{I}$ be the set of indices $i \in [n-1]$ for which $x_k = w_{j^*,k}$ for some $k \in [f]$. Set $\pi \leftarrow \{x_i\}_{i \in \mathcal{I}}$.
6. Output $(\mathcal{I}, \pi)$

Fig. 4: The Tracing Algorithm for the Traceable Bottom-Up Secret Sharing TBUSS.

4 Non-Imputability of TBUSS

Let us recall the Non-Imputability game defined in section 2.3. To achieve non-imputability in Traceable Bottom-Up Secret Sharing (TBUSS), we enhance the scheme to prevent malicious tracers from falsely accusing honest parties. The original construction (Definition 3 and Section 3.1) exposes the random field elements x_j in the tracing key tk and verification key vk, allowing adversaries to trivially impute any honest party by including their x_j in a forged proof. To address this, we use a one-way function $\mathcal{F} : \mathbb{F} \to \mathbb{F}$. Intuitively, we "hide" each party's random evaluation point x_j behind a one-way function, yet still allow any extracted root to be linked back to its owner. The enhanced scheme, Non-Imputable TBUSS (NITBUSS), modifies the algorithms as follows.

4.1 Modified Algorithms for NITBUSS

In the following, we describe the Share, Trace and Verify algorithms and skip the Recon algorithm as it is easy to write following the Share algorithm.

1. $\mathsf{Share}(s, \boldsymbol{\sigma}_B, B)$
   - Define a polynomial q of degree $n-1$ over $\mathbb{F}$ such that
     - $q(0) = s$
     - Parse $\boldsymbol{\sigma}_B \leftarrow \{(x_j, \sigma_j)\}_{j \in B}$
     - Sample $x_{-1}, x_{-2}, \ldots, x_{-(n-t-1)} \xleftarrow{\$} \mathbb{F} \setminus \{x_j\}_{j \in B}$.
     - For all $j \in B$ and set $q(x_j) := \sigma_j$
   - Compute $\varphi := (\varphi_1, \varphi_2, \ldots, \varphi_{n-t-1})$, where $\varphi_i = (x_{-i}, q(x_{-i}))$.
   - For each $j \in [n-1]$ (corresponding parties in B):
     - Compute $u_j = \mathcal{F}(x_j)$
   - Set $\mathsf{tk} = (u_1, u_2, \ldots, u_{n-1})$ and $\mathsf{vk} = (u_1, u_2, \ldots, u_{n-1})$.
   - Output $(\varphi, \mathsf{tk}, \mathsf{vk})$
2. $\mathbf{Trace}^{\mathcal{R}}_{\mathsf{NITBUSS}}(\mathsf{tk}, 1^{1/\epsilon})$
   - Parse tk as $(u_1, u_2, \ldots u_n)$.
   - Run the Trace algorithm to obtain a list of candidate polynomials $G = \{g_j\}_j$ from list decoding. For each candidate polynomial g_j, do:
     - Factor g_j to extract roots $w_1, \ldots, w_f$.
     - For each root w_i:
       - Compute $u'_i = \mathcal{F}(w_i)$
       - If $u'_i \notin \{u_1, \ldots, u_f\}$, discard g_j.
   - If candidates remain, take the first valid $g^* \in G$ with roots $w^*_1, \ldots, w^*_f$.
   - For each w^*_i, find the index $k \in [n-1]$ such that $\mathcal{F}(w^*_i) = u_k$ and add k to $\mathcal{I}$.
   - Set $\pi \leftarrow (w^*_1, \ldots, w^*_f)$.
   - Output $(\mathcal{I}, \pi)$.
3. $\mathbf{Verify}(\mathsf{vk}, \mathcal{I}, \pi)$
   - Parse $\mathsf{vk} = (u_1, u_2, \ldots, u_{n-1})$, $\mathcal{I} = \{i_1, i_2, \ldots, i_f\} \subset [n-1]$, and $\pi = (w_1, \ldots, w_f)$
   - For each $k \in [f]$:
     - Check $\mathcal{F}(w_k) = u_{i_k}$
   - Output 1 if all checks pass; else, output 0.

The following theorem formally establishes the advantage of non-imputability of $\mathsf{NITBUSS}$. We omit the proof and include it in the full version of this paper.

Theorem 3 (Non-Imputability of $\mathsf{NITBUSS}$). *Let $\mathcal{F}$ be a one-way function. For every PPT adversary $\mathcal{A}$ winning the non-imputability game for $\mathsf{NITBUSS}$, there exists a PPT algorithm (inverter) $\mathcal{B}$ such that for every $\lambda \in \mathbb{N}$ it holds that,*

$$\mathsf{Adv}^{\mathsf{ni}}_{\mathcal{A},\mathsf{NITBUSS}}(\lambda) = \Pr[\mathcal{F}(\mathcal{B}(\mathcal{F}(x^*))) = \mathcal{F}(x^*)]$$

where $x \xleftarrow{\$} \mathbb{F}$ and the probability is also over the random coins of $\mathcal{B}$.

5 Traceable Social Key Recovery

We propose a traceable version of the SKR scheme. The tracer has access to reconstruction boxes $\mathcal{R}$, which produce outputs upon receiving inputs in a specified format. In a $(t+1)$-out-of-$(n-1)$ scheme, if there are $f < t+1$ corruptions, the adversary may have hard-coded f inputs into the reconstruction box. In such a case, the tracer must provide the remaining $t+1-f$ valid inputs to complete the reconstruction and obtain the output. Suppose there are N parties, and let $\mathcal{U}$ be an access structure consisting of pairs of sets (B, R) such that $B \subseteq [N]$ and $R \subseteq B$ with $|B| = n-1$ and $|R| = t+1$. The TSKR scheme consists of three protocols, i.e., $\mathsf{TSKR} = (\Pi_{\mathsf{Init}}, \Pi_{\mathsf{Back}}, \Pi_{\mathsf{Rec}})$. In the Π_{Init} protocol, each party runs a key generation algorithm to produce a key pair $(\mathsf{sk}_i, \mathsf{pk}_i)$. In the Π_{Back} protocol, each party backs up their public value φ_i. Once all parties have executed both the Π_{Init} and Π_{Back} protocols, they can invoke the recovery protocol Π_{Rec} any number of times to reconstruct the secret. Note that Π_{Back} and Π_{Rec} invoke $\mathsf{TBUSS.Share}$ and $\mathsf{TBUSS.Recon}$, respectively.

Traceable Social Key Recovery (TSKR)

$\Pi_{\mathsf{Init}}(1^\lambda, 1^N)$: Each party P_i runs the key generation algorithm $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and publishes their public key pk_i and stores their secret key sk_i.

$\Pi_{\mathsf{Back}}(i, B, \mathbf{pk}_B, \mathbf{sk}_{R\cup\{i\}})$: Party P_i runs this as a key-owner with a set of $(n-1)$ guardians $\{P_j\}_{j\in B}$ as follows:

- For each guardian P_j, P_i sends a backup request to P_j.
- Each P_j computes $x_{i,j} := H(j, \mathsf{sk}_j, \mathsf{pk}_i)$ and $\sigma_{i,j} = H(i, \mathsf{sk}_j)$ and sends $(x_j, \sigma_{i,j})$ back to the party P_i.
- On receiving $(x_{i,j}, \sigma_{i,j})$, and P_i defines $s := \mathsf{sk}_i$ computes:

$$(\varphi, \mathsf{tk}, \mathsf{vk}) \leftarrow \mathsf{TBUSS.Share}(\mathsf{sk}_i, \{(x_{i,j}, \sigma_{i,j})\}_{j\in B}, B)$$

  and publishes $pub := \varphi$.

$\Pi_{\mathsf{Rec}}(i, R, \mathbf{pk}_{[N]}, \varphi, \mathbf{sk}_R)$: The party P_i wants to compute his secret with a set of $t+1$ guardians as follows:

- For each guardian P_j, P_i send a request to P_j for $j \in R$.
- After receiving $(x_{i,j}, \sigma_{i,j})_{j\in R}$, P_i retrieve φ and compute

$$s' \leftarrow \mathsf{TBUSS.Recon}(\varphi, \{(x_{i,j}, \sigma_{i,j})\}_{j\in R}, R)$$

- Then the party P_i checks the validity of the key pair (s', pk_i). If yes then privately output $\mathsf{sk}_i = s'$, else output $\perp$.

$\mathsf{Game.Trace}_{\mathcal{A},\mathsf{TSKR},\varepsilon}(\lambda, N)$**: Tracing game for** TSKR**.**

1. $\mathcal{A}(1^\lambda, 1^N, n-1, t+1, f)$ outputs $(\{\mathcal{I}_k\}_{k\in[N]}, \mathsf{state})$, where $\mathcal{I}_k \subset [n-1]$ and $|\mathcal{I}_k| = f(< t+1)$ is the set of parties to corrupt.
2. N secrets $\{s_k\}_{k\in[N]}$ is sampled uniformly at random from $\mathcal{S}$.
3. Run: $\mathsf{Share}(s_k, \boldsymbol{\sigma}_{B_k}, B)$ output $(\varphi_k, \mathsf{tk}^{(k)}, \mathsf{vk}^{(k)})$ for $k \in [N]$.
4. On input all shares of parties in $\mathcal{I}_1, \ldots, \mathcal{I}_N$, the adversary $\mathcal{A}(\mathsf{state}, \boldsymbol{\sigma}_{\mathcal{I}_1}, \ldots, \boldsymbol{\sigma}_{\mathcal{I}_N})$ outputs reconstruction boxes $\mathcal{R}_1, \ldots, \mathcal{R}_N$ respectively.
5. $\mathsf{Trace}^{\mathcal{R}_1,\ldots,\mathcal{R}_N}(\mathsf{tk}^{(1)}, \ldots, \mathsf{tk}^{(N)})$ outputs $(\mathcal{I}'_1, \ldots, \mathcal{I}'_N, \pi_1, \ldots, \pi_N)$.
6. $\mathcal{A}$ wins if: $\mathcal{R}_1, \ldots, \mathcal{R}_N$ reconstruct the secret from consistent inputs with probability at least ε, i.e., $\mathcal{R}_1, \ldots, \mathcal{R}_N$ are good and there exists $k \in [N]$ such that either $\mathcal{I}_k \neq \mathcal{I}'_k$ or $\mathsf{Verify}(\mathsf{vk}^{(k)}, \mathcal{I}'_k, \pi_k) = 0$.

Fig. 5: The Tracing Game for the Traceable Social Key Recovery TSKR.

5.1 Security of TSKR

See the Traceability game for SKR in Figure 6. This game is slightly different from the tracing game we have defined for BUSS in Figure 1. The adversary can corrupt $f < t+1$ guardians for every party P_k for $k \in [N]$. Let s_k be the secret key of party P_k. Now, the adversary can corrupt f guardians of party P_k to gain access to their shares σ_{k,i_j} for all $j \in [f]$ and outputs a reconstruction box $\mathcal{R}_k$ for every $k \in [N]$. The Tracer with access to $\mathcal{R}_k$ for $k \in [N]$ and with the corresponding guardians' trace key tuple $\mathsf{tk}^{(k)}$ for $k \in [N]$ outputs the list of corrupt guardians for party P_k for $k \in [N]$. Note that there is a master trace key tk for all the participating N parties, which the parties can derive by themselves. The trace key vectors $\mathsf{tk}^{(k)}$ are subsets of this master trace key tk. The parties

Definition 8 (Traceability of TSKR**).** *Let* $\mathsf{TSKR} = (\Pi_{\mathsf{Init}}, \Pi_{\mathsf{Back}}, \Pi_{\mathsf{Rec}})$ *be a Traceable SKR. Let* $\varepsilon = \varepsilon(\lambda, N)$ *be a function of the security parameters. We say that* TSKR *satisfies traceability if for every PPT adversary* $\mathcal{A}$*, the following function is negligible in* λ *and* N*:*

$$\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TSKR},\epsilon}(\lambda, N) := \Pr\left[\mathsf{Game.Trace}_{\mathcal{A},\mathsf{TSKR},\varepsilon}(\lambda, N) = 1\right].$$

5.2 Tracing on SKR

First, we describe the corruption model used in our SKR scheme. Our model is similar to that of Boneh, Partap, and Rotem [9], with the key difference being that, in our setting, each party holds its own secret key sk_i, rather than a single shared secret. Any subset $\mathcal{I} \subset B$ of f colluding parties can construct a reconstruction box $\mathcal{R}$ such that, when given any additional $t-f+1$ valid inputs, it can reconstruct the corresponding secret. We consider N such colluding groups, each constructing its own reconstruction box $\mathcal{R}_1, \ldots, \mathcal{R}_N$. For simplicity, we

Algorithm: $\mathsf{Trace}^{\mathcal{R}^{(1)},\ldots,\mathcal{R}^{(N)}}_{\mathsf{TSKR}}(\mathsf{tk}, f)$

1. Parse tk as $(\mathsf{tk}^{(1)}, \ldots, \mathsf{tk}^{(N)})$.
2. For $k = 1, \ldots, N$:
   - With parameters $M, C \in \mathbb{N}$ and $\epsilon_1, \ldots, \epsilon_N$ run $\mathsf{Trace}^{\mathcal{R}_k}_{\mathsf{TBUSS}}(\mathsf{tk}^{(k)}) \to (\mathcal{I}_k, \pi_k)$
3. Compute $\mathcal{I}_{\mathsf{TSKR}} = (\mathcal{I}_1, \ldots, \mathcal{I}_N)$ and $\pi = (\pi_1, \ldots, \pi_k)$
4. Output $(\mathcal{I}_{\mathsf{TSKR}}, \pi)$

Fig. 6: The Tracing Algorithm for TSKR.

first assume that each colluding group consists of exactly f parties. Given any reconstruction box $\mathcal{R}^{(k)}$, if it receives $t - f + 1$ valid inputs, it can recover the associated secret. A tracer equipped with the trace key tk (as in Figure 5) can then identify the colluding parties. So for tracing in the SKR scheme, the tracing algorithm runs over all the reconstruction boxes $\mathcal{R}_1, \ldots, \mathcal{R}_N$. After the tracing on each reconstruction box, we get the colluding sets are $\mathcal{I}_1, \ldots, \mathcal{I}_N$ then the overall colluding set is $\mathcal{I} = \cup_{k=1}^{N} \mathcal{I}_k$. We present our tracing algorithm for TSKR in Figure 6.

Theorem 4. *Let* TSKR *be the Traceable Social Key Recovery (SKR) scheme defined in Section 5.2. For every adversary* $\mathcal{A}$*, security parameters* $\lambda, N \in \mathbb{N}$*, parameters* $M, C \in \mathbb{N}$*, and* $\epsilon_1, \epsilon_2, \ldots, \epsilon_N \in \left[\sqrt{\frac{2(M+n+t^2-ft)}{p}}, 1\right]$ *with* $\epsilon_i > \sqrt{2C/M}$ *for each* $i \in [N]$ *and* $\sqrt{fM} \leq C \leq M < p$*, we have:*

$$\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TSKR},\epsilon}(\lambda, N) \leq N \cdot \left(e^{\frac{-\epsilon^2 M}{2} \cdot (1-1/r)^2} + \frac{f \cdot (n-f-1) \cdot \tau}{p} \right)$$

where $\epsilon = \min\{\epsilon_1, \ldots, \epsilon_N\}$*,* $|\mathbb{F}| = p$ *(field size),* $r = \dfrac{\epsilon^2 M}{2C}$*,* $n = n(\lambda)$ *(size of guardian set* $B = n - 1$*),* $f = f(\lambda)$ *(number of corruptions,* $f < t + 1$*),* $\tau = \tau(M, f, \log p)$ *be the list size from the Guruswami-Sudan Algorithm* $\mathcal{D}_{\mathsf{GS}}$*.*

Proof. It is easy to see that $\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TSKR},\epsilon}(\lambda, N) \leq N \cdot \mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TBUSS},\epsilon}(\lambda)$ and by Theorem 2 we have that $\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TBUSS},\epsilon}(\lambda) \leq \left(e^{\frac{-\epsilon^2 M}{2} \cdot (1-1/r)^2} + \dfrac{f \cdot (n-f-1) \cdot \tau}{p} \right)$. Thus, we have the theorem. □

Remark 1. Note that for each $k \in [N]$ we have that $|\mathcal{I}_k| = f < t + 1$. But it is not true that $|\mathcal{I}_{\mathsf{TSKR}}| = f$ or $|\mathcal{I}_{\mathsf{TSKR}}| < t + 1$. We do not have any restriction on the size of $\mathcal{I}_{\mathsf{TSKR}}$ other than the maximum upper bound of N. It may very well happen that $|\mathcal{I}_{\mathsf{TSKR}}| = N$, i.e., all the parties $P_1, \ldots, P_N$ participating in TSKR protocol are corrupt. Each party P_i can be a guardian of several other parties, thus P_i can freely choose whose share to sell, i.e., the number of corruption of P_i can be from the set $\{0, \ldots, N-1\}$. So, for the corruption model of TSKR, we do

not a priori declare a number of corruption, unlike TBUSS, in which the number of corruption f, is known beforehand. We also have the provision to report every set of corrupt parties $\mathcal{I}_k$ for each party P_k where $k \in [N]$, to track the corrupt guardians.

5.3 Non-Imputability of SKR

We describe the NISKR construction in the full version of this paper; however, we state the formal result on non-imputability in this section.

Theorem 5 (Non-Imputability of NISKR). *Let $\mathcal{F}$ be a one-way function. For every PPT adversary $\mathcal{A}$ winning the non-imputability game for NISKR, there exists a PPT algorithm (inverter) $\mathcal{B}$ such that for every $\lambda, N \in \mathbb{N}$ it holds that,*

$$\mathsf{Adv}^{\mathsf{ni}}_{\mathcal{A},\mathsf{NISKR}}(\lambda, N) = \Pr[\mathcal{F}(\mathcal{B}(\mathcal{F}(x^*))) = \mathcal{F}(x^*)]$$

where $x \xleftarrow{\$} \mathbb{F}$ and the probability is also over the randomness $\mathcal{B}$.

The proof of the Theorem 5 is similar to the proof of the Theorem 7, hence provided in the full version of the paper.

6 Complexity and Comparison of Our Tracing Procedures

We now provide a comparative overview of existing traceable secret sharing (TSS) schemes. The scheme of Goyal, Song, and Srinivasan (GSS) [28] is based on Shamir's secret sharing and was the first to achieve public traceability. However, its practicality is limited by the fact that the share size grows by a factor of λ, making it unsuitable for large-scale deployments. Moreover, the scheme does not support weighted access structures, and its security guarantees do not extend against computationally unbounded adversaries.

Boneh, Partap, and Rotem [9] subsequently introduced traceable variants of Shamir's and Blakley's schemes. These constructions are considerably more efficient: the share size only doubles compared to the original schemes, and they work for arbitrary thresholds t. Nonetheless, their notion of traceability is private rather than public, and they do not extend to weighted access structures or offer security in the presence of unbounded adversaries.

The Mignotte-based traceable scheme (MTSS) [30] achieves the traceability notion of [9], the share size again grows only by a factor of 2, but the tracing algorithm is computationally demanding, running in time exponential in κ.
Our basic traceability notion for TBUSS satisfies the traceability guarantee introduced in the work of Boneh, Partap, and Rotem [9]. Also, this is efficient in the sense that the share size is increased by only a factor of 2. The tracing algorithm for TBUSS is also efficient as it matches the exact time complexity of the Traceable Shamir in the work of [9]. The SKR scheme has tracing complexity of $N \cdot \mathsf{poly}(\lambda, \epsilon^{-1})$ and follows the same efficiency except for a multiplicative factor of N.

TSS scheme	**$\|\mathsf{sh}\|$ increase**	**Trace Time**	**Public**	**Any $\mathcal{A}$**
GSS [28]	λ	$\text{poly}(\lambda, \epsilon^{-1})$	yes	no
T-Shamir [9]	2	$\text{poly}(\lambda, \epsilon^{-1})$	no	no
T-Blakely [9]	2	$\text{poly}(\lambda, \epsilon^{-1})$	no	no
NI-Shamir [9]	2	$\text{poly}(\lambda, \epsilon^{-1})$	no	no
NI-Blakely [9]	2	$\text{poly}(\lambda, \epsilon^{-1})$	no	no
MTSS [30]	2	$\text{poly}(2^{\kappa}, \epsilon^{-1})$	no	no
SPMTSS [30]	2	$\text{poly}(\lambda, \epsilon^{-1})$	semi	yes
MTSS for small t [30]	$2\lambda/t$	$\text{poly}(2^{\kappa}, \epsilon^{-1})$	no	no
SPMTSS for small t [30]	$2\lambda/t$	$\text{poly}(\lambda, \epsilon^{-1})$	semi	yes
TBUSS [This Work]	2	$\text{poly}(\lambda, \epsilon^{-1})$	no	no
TSKR [This Work]	2	$N \cdot \text{poly}(\lambda, \epsilon^{-1})$	no	no
NITBUSS [This Work]	2	$\text{poly}(\lambda, \epsilon^{-1})$	no	no
NISKR [This Work]	2	$N \cdot \text{poly}(\lambda, \epsilon^{-1})$	no	no

Table 1: Overview of traceable secret sharing schemes. Here, $|\mathsf{sh}|$ denotes the share size. The column 'Any $\mathcal{A}$' specifies whether security is guaranteed against computationally unbounded adversaries.

7 Conclusion and Future Direction

In this paper, we have proposed a tracing mechanism for the Bottom-Up Secret Sharing (BUSS) scheme. We have thoroughly analyzed the tracing advantage of BUSS. Additionally, we have shown Non-Imputability of BUSS by integrating the construction with a one-way function. Eventually, we have shown that the hardness of inversion of the one-way function yields non-imputability security of BUSS. As an application of traceable BUSS we have proposed a traceable Social Key Recovery (TSKR) construction and provided a tracing mechanism along with non-imputability advantage. Although our Tracing Mechanism for TSKR essentially relies on the tracing mechanism of BUSS, it eventually contributes a factor of N in the tracing advantage probability. Also we have shown non-imputability of both BUSS and SKR, thereby constructing non-impitable variants of the traceable schemes BUSS and SKR, namely NITBUSS and NISKR.

In this work we have shown private traceability of BUSS and SKR, only the tracer learns the trace keys of the parties and hence can perform trace. An interesting avenue for future work is public traceability of BUSS. Also, it will be an interesting future direction to come up with a tracing mechanism for SKR independent of the tracing mechanism for BUSS.

References

1. Argent: How to recover my wallet with guardians on-chain (com- plete guide) (2025). https://doi.org/https://support.argent.xyz/hc/en-us/articles/

360007338877-How-to-recover-my-wallet-with-guardians-onchain-complete-guide
2. Baghery, K., Ebrahimi, E., Mirzamohammadi, O., Sedaghat, M.: Traceable verifiable secret sharing and applications. Cryptology ePrint Archive, Paper 2025/318 (2025), https://eprint.iacr.org/2025/318
3. Baird, L., Garg, S., Jain, A., Mukherjee, P., Sinha, R., Wang, M., Zhang, Y.: Threshold signatures in the multiverse. Cryptology ePrint Archive, Paper 2023/063 (2023), https://eprint.iacr.org/2023/063
4. Bellare, M., Dai, W., Rogaway, P.: Reimagining secret sharing: Creating a safer and more versatile primitive by adding authenticity, correcting errors, and reducing randomness requirements. Cryptology ePrint Archive, Paper 2020/800 (2020), https://eprint.iacr.org/2020/800
5. Bhattacharyya, R., Bormet, J., Faust, S., Mukherjee, P., Othman, H.: CCA-secure traceable threshold (ID-based) encryption and application. Cryptology ePrint Archive, Paper 2025/341 (2025), https://eprint.iacr.org/2025/341
6. Blakley, G.R.: Safeguarding cryptographic keys. 1979 International Workshop on Managing Requirements Knowledge (MARK) pp. 313–318 (1899), https://api.semanticscholar.org/CorpusID:38199738
7. Boneh, D., Franklin, M.: An efficient public key traitor tracing scheme. In: Wiener, M. (ed.) Advances in Cryptology — CRYPTO' 99. pp. 338–353. Springer Berlin Heidelberg, Berlin, Heidelberg (1999)
8. Boneh, D., Naor, M.: Traitor tracing with constant size ciphertext. In: Proceedings of the 15th ACM Conference on Computer and Communications Security. p. 501–510. CCS '08, Association for Computing Machinery, New York, NY, USA (2008). https://doi.org/10.1145/1455770.1455834, https://doi.org/10.1145/1455770.1455834
9. Boneh, D., Partap, A., Rotem, L.: Traceable secret sharing: Strong security and efficient constructions. Cryptology ePrint Archive, Paper 2024/405 (2024), https://eprint.iacr.org/2024/405
10. Boneh, D., Partap, A., Rotem, L.: Traceable verifiable random functions. Cryptology ePrint Archive, Paper 2025/312 (2025), https://eprint.iacr.org/2025/312
11. Boneh, D., Sahai, A., Waters, B.: Fully collusion resistant traitor tracing with short ciphertexts and private keys. In: Vaudenay, S. (ed.) Advances in Cryptology - EUROCRYPT 2006. pp. 573–592. Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
12. Boneh, D., Shaw, J.: Collusion-secure fingerprinting for digital data. In: Coppersmith, D. (ed.) Advances in Cryptology — CRYPT0' 95. pp. 452–465. Springer Berlin Heidelberg, Berlin, Heidelberg (1995)
13. Boneh, D., Zhandry, M.: Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. Algorithmica **79**(4), 1233–1285 (Dec 2017). https://doi.org/10.1007/s00453-016-0242-8, https://doi.org/10.1007/s00453-016-0242-8
14. Bormet, J., Dziembowski, S., Faust, S., Lizurej, T., Mielniczuk, M.: Strong secret sharing with snitching. Cryptology ePrint Archive, Paper 2025/1119 (2025), https://eprint.iacr.org/2025/1119
15. Bormet, J., Hofmann, J., Othman, H.: Traceable threshold encryption without trusted dealer. Cryptology ePrint Archive, Paper 2025/342 (2025), https://eprint.iacr.org/2025/342
16. Buterin, V.: Why we need wide adoption of social recovery wallets. (2021). https://doi.org/https://vitalik.eth.limo/general/2021/01/11/recovery.html

17. Chatzigiannis, P., Chalkias, K., Kate, A., Mangipudi, E.V., Minaei, M., Mondal, M.: SoK: Web3 recovery mechanisms. Cryptology ePrint Archive, Paper 2023/1575 (2023), https://eprint.iacr.org/2023/1575
18. Chen, Y., Vaikuntanathan, V., Waters, B., Wee, H., Wichs, D.: Traitor-tracing from lwe made simple and attribute-based. In: Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part II. p. 341–369. Springer-Verlag, Berlin, Heidelberg (2018). https://doi.org/10.1007/978-3-030-03810-6_13, https://doi.org/10.1007/978-3-030-03810-6_13
19. Chor, B., Fiat, A., Naor, M., Pinkas, B.: Tracing traitors. IEEE Transactions on Information Theory **46**(3), 893–910 (2000). https://doi.org/10.1109/18.841169
20. Dodis, Y., Fazio, N.: Public key trace and revoke scheme secure against adaptive chosen ciphertext attack. Cryptology ePrint Archive, Paper 2003/095 (2003), https://eprint.iacr.org/2003/095
21. Dziembowski, S., Faust, S., Lizurej, T., Mielniczuk, M.: Secret sharing with snitching. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 840–853. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3658644.3690296, https://doi.org/10.1145/3658644.3690296
22. Farràs, O., Guiot, M.: Traceable secret sharing schemes for general access structures. Cryptology ePrint Archive, Paper 2025/1120 (2025), https://eprint.iacr.org/2025/1120
23. Fiat, A., Tassa, T.: Dynamic traitor training. In: Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings. Lecture Notes in Computer Science, vol. 1666, pp. 354–371. Springer (1999). https://doi.org/10.1007/3-540-48405-1_23
24. Garg, S., Kumarasubramanian, A., Sahai, A., Waters, B.: Building efficient fully collusion-resilient traitor tracing and revocation schemes. In: Proceedings of the 17th ACM Conference on Computer and Communications Security. p. 121–130. CCS '10, Association for Computing Machinery, New York, NY, USA (2010). https://doi.org/10.1145/1866307.1866322, https://doi.org/10.1145/1866307.1866322
25. Goldreich, O.: Foundations of Cryptography. Cambridge University Press (2001)
26. Gong, T., Kate, A., Maji, H.K., Nguyen, H.H.: Disincentivize collusion in verifiable secret sharing. In: Fehr, S., Fouque, P.A. (eds.) Advances in Cryptology – EUROCRYPT 2025. pp. 34–64. Springer Nature Switzerland, Cham (2025)
27. Goyal, R., Koppula, V., Waters, B.: Collusion resistant traitor tracing from learning with errors. In: Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing. p. 660–670. STOC 2018, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3188745.3188844, https://doi.org/10.1145/3188745.3188844
28. Goyal, V., Song, Y., Srinivasan, A.: Traceable secret sharing and applications. In: Advances in Cryptology – CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part III. p. 718–747. Springer-Verlag, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-030-84252-9_24, https://doi.org/10.1007/978-3-030-84252-9_24
29. Guruswami, V.: Improved decoding of reed-solomon and algebraic-geometric codes. vol. 45, pp. 28–37 (12 1998). https://doi.org/10.1109/SFCS.1998.743426

30. Hoffmann, C.: Traceable secret sharing based on the chinese remainder theorem. Cryptology ePrint Archive, Paper 2024/811 (2024), https://eprint.iacr.org/2024/811

31. Kate, A., Mukherjee, P., Saleem, H., Sarkar, P., Roberts, B.: ANARKey: A new approach to (socially) recover keys. Cryptology ePrint Archive, Paper 2025/551 (2025), https://eprint.iacr.org/2025/551

32. Kiayias, A., Yung, M.: Self protecting pirates and black-box traitor tracing. In: Kilian, J. (ed.) Advances in Cryptology — CRYPTO 2001. pp. 63–79. Springer Berlin Heidelberg, Berlin, Heidelberg (2001)

33. Kurosawa, K., Desmedt, Y.: Optimum traitor tracing and asymmetric schemes. In: Advances in Cryptology - EUROCRYPT '98, International Conference on the Theory and Application of Cryptographic Techniques, Espoo, Finland, May 31 - June 4, 1998, Proceeding. Lecture Notes in Computer Science, vol. 1403, pp. 145–157. Springer (1998). https://doi.org/10.1007/BFb0054123

34. Lindell, Y.: Cryptography and mpc in coinbase wallet as a service (waas). (2023). https://doi.org/https://www.coinbase.com/en-in/blog/digital-asset-management-with-mpc-whitepaper

35. Little, R., Qin, L., Varia, M.: Secure account recovery for a privacy-preserving web service. Cryptology ePrint Archive, Paper 2024/962 (2024), https://eprint.iacr.org/2024/962

36. Mignotte, M.: How to share a secret. In: Beth, T. (ed.) Cryptography. pp. 371–375. Springer Berlin Heidelberg, Berlin, Heidelberg (1983)

37. Naor, M., Pinkas, B.: Threshold traitor tracing. In: Krawczyk, H. (ed.) Advances in Cryptology — CRYPTO '98. pp. 502–517. Springer Berlin Heidelberg, Berlin, Heidelberg (1998)

38. Pedin, A.B., Siasi, N., Sameni, M.: Smart contract-based social recovery wallet management scheme for digital assets. In: Proceedings of the 2023 ACM Southeast Conference. p. 177–181. ACMSE '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3564746.3587016, https://doi.org/10.1145/3564746.3587016

39. Shamir, A.: How to share a secret. Commun. ACM **22**(11), 612–613 (Nov 1979). https://doi.org/10.1145/359168.359176, https://doi.org/10.1145/359168.359176

40. Times, T.N.Y.: Tens of billions worth of bitcoin have been locked by people who forgot their key. (2021). https://doi.org/https://www.nytimes.com/2021/01/13/business/tens-of-billions-worth-of-bitcoin-have-been-locked-by-people-who-forgot-their-key.html

41. Wee, H.: Functional encryption for quadratic functions from k-lin, revisited. In: Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part I. p. 210–228. Springer-Verlag, Berlin, Heidelberg (2020). https://doi.org/10.1007/978-3-030-64375-1_8, https://doi.org/10.1007/978-3-030-64375-1_8

42. Zhandry, M.: New techniques for traitor tracing: Size $n^{1/3}$ and more from pairings. Cryptology ePrint Archive, Paper 2020/954 (2020), https://eprint.iacr.org/2020/954

A Paradigm of BUSS, SKR [31]

We follow the definitions and notations from the work of [31] and explain the model of SKR and BUSS. Let us consider N parties, denoted by $P_1, P_2, \ldots, P_N$, who are pairwise connected by secure and authenticated communication channels. Additionally, there exists a reliable public storage medium, such as a blockchain bulletin board, where immutable public values may be posted. Each party P_i generates its own key pair $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda)$, where λ is the security parameter. Public keys pk_i serve as permanent identities and are posted to the bulletin board.

A global threshold t is fixed. Each party backs up her secret key sk_i using exactly $n > t$ guardians (with $n < N$). Importantly, although the description assumes a uniform n, the protocol also works when each party selects a different n, provided the requirement $n > t$ holds. Thus the framework is flexible across community members.

Challenge with Standard Secret Sharing. In a traditional t-out-of-n Shamir secret sharing, a key-owner P_i splits her secret key sk_i into n shares $\sigma_{i,1}, \ldots, \sigma_{i,n}$. Each guardian receives one share, which must be stored securely and reliably. However, in the community setting, where every member may simultaneously be a key-owner and a guardian, this leads to scalability issues: a single guardian P_j may need to manage m different shares from different key-owners. This implies $O(m)$ storage requirements, which quickly becomes a practical bottleneck.

The main innovation in the work of [31] is to design a scheme where guardians need not store anything beyond their own secret key sk_j. Shares for key recovery are deterministically derived from a guardian's secret, eliminating the storage overhead. The protocol exploits the fact that each member already maintains a secret key securely. We describe the *Backup* and *Recovery* protocols in [31] as follows.

- *Backup protocol.* Suppose P_i wishes to backup her key sk_i. She selects a guardian set $B \subseteq [N] \setminus \{i\}$ of size $n-1$. In a standard approach, a share $\sigma_{i,j}$ would be sampled independently and given to each guardian P_j. In the work of [31], instead, the shares are derived directly from the guardian's own secret:

$$\sigma_{i,j} := H(i, \mathsf{sk}_i) \text{ or equivalently } H(\mathsf{pk}_i, \mathsf{sk}_j)$$

  where H is modeled as a random oracle. This guarantees that $\sigma_{i,j}$ reveals no information about sk_j, since recovering sk_j from pk_j is assumed to be computationally hard (e.g., due to the discrete logarithm assumption in BLS or ECDSA signatures). The derivation ensures that guardians do not need to store additional data: when needed, they can recompute $\sigma_{i,j}$ on the fly. Once P_i collects $\{\sigma_{i,j}\}_{j \in B}$, she constructs a polynomial q_i of degree $n-1$ over a suitable field:

$$q_i(0) = \mathsf{sk}_i, \quad q_i(j) = \sigma_{i,j} \quad \forall j \in B$$

This defines q_i uniquely. However, this results in an n-out-of-n scheme, since reconstructing q_i requires all n points. To reduce the threshold to $(t+1)$, P_i publishes additional evaluations following a technique stated in the work of [3], as follows,

$$\varphi_i = (q_i(-1), q_i(-2), \ldots, q_i(-(n-t-1)))$$

These public values reduce the effective reconstruction threshold, since only $(t+1)$ guardian shares are then needed along with the $(n-t-1)$ public points. Importantly, the additional points are computed at negative evaluation indices $\{-1, -2, \ldots, -(n-t-1)\}$, following a technique by Baird et al. [3], ensuring independence from the guardian inputs. Thus, the backup protocol can be summarized:

1. P_i chooses a guardian B.
2. Each guardian $P_j \in B$ computes $\sigma_{i,j} := H(i, \mathsf{sk}_j)$ ans sends it to P_i.
3. P_i interpolates q_i with conditions above.
4. P_i publishes $(n-t-1)$ public points φ_i on the bulletin board.

– *Recovery Protocol.* If P_i looses her secret key sk_i, she can initiate recovery with any subset $R \subseteq B$ of size $t+1$.
  1. P_i contacts guardian $P_j \in R$.
  2. Each guardian recomputes $\sigma_{i,j} = H(i, \mathsf{sk}_j)$ and sends it back.
  3. P_i now holds $(t+1)$ shares from R along with the public evaluations φ_i.
  4. Using interpolation, she reconstructs q_i and recovers $\mathsf{sk}_i = q_i(0)$.
  5. Finally, she checks whether sk_i corresponds to the known pk_i.

  This guarantees correctness: with $(t+1)$ guardian responses and the $(n-t-1)$ public points, there are exactly n evaluation points of the degree-$(n-1)$ polynomial q_i, which uniquely determine it.

Security Properties. If at most t guardians are corrupted, the adversary learns at most t evaluations of q_i. Together with the $(n-t-1)$ public evaluations, this gives at most $n-1$ points, insufficient to interpolate q_i and recover sk_i. Thus, secrecy is preserved.

– *Independence of Shares.* If a guardian P_j acts for multiple key-owners $P_{i_1}, \ldots, P_{i_m}$, her shares are:

$$\sigma_{i_1,j} = H(i_1, \mathsf{sk}_j), \ldots, \sigma_{i_m,j} = H(i_m, \mathsf{sk}_j)$$

  Under the random oracle assumption, these outputs are uncorrelated and reveal no information about sk_i.
– *Collusion Resistance.* The set of guardians B chosen by a key-owner is not public. This reduces the risk of collusion among guardians, since adversaries cannot easily identify which parties need to be compromised simultaneously.
– *Malicious Security.* If a malicious guardian provides inconsistent values during backup or recovery, the final interpolation may fail. However, correctness verification against pk_i ensures that faulty reconstructions are detected,

resulting in an abort rather than a silent compromise. While the scheme does not yet support identifiable aborts (i.e., pinpointing the misbehaving guardian), it guarantees that malicious actions cannot result in incorrect key recovery.

– *Adaptive Security.* The scheme is designed against adaptive adversaries who can corrupt parties dynamically during protocol execution. For example, if a simulator generates a share response for an honest guardian P_j without knowing sk_j, and later knowing P_j becomes corrupted, consistency must be preserved. This is achieved by programming the random oracle so that $\sigma_{i,j} = H(i, \mathsf{sk}_j)$ always holds retrospectively. Similarly, if a key-owner P_i is corrupted after backup, the simulator ensures that the public points remain consistent with sk_i. This mechanism formalizes a new variant of secret sharing called *bottom-up secret sharing* (BUSS), which ensures perfect adaptive simulation security.

B Proof of Theorem 2

Proof. Consider an adversary $\mathcal{A}$ in the traceability experiment $\mathsf{Game.Trace}_{\mathcal{A},\mathsf{TBUSS},\varepsilon}(\lambda)$. Let $\mathcal{I}$ be the corrupted set of size f with shares $\{(x_j^*, \sigma_j^*)\}_{j=1}^{f}$, and let φ be the public values defining $h_\varphi(X) = \prod_{k=-1}^{-(n-t-1)} \frac{x_k - X}{x_k}$. The tracing key is $\mathsf{tk} = \{x_i\}_{i=1}^{n-1}$. Let G be the event that the adversary outputs a reconstruction box $\mathcal{R}$ that is $(n, t, \boldsymbol{y}, \epsilon)$-good where n and t are chosen by the adversary and $\boldsymbol{y}$ is given by the challenger. Conditioned on $\neg\mathsf{G}$, the output of the experiment is 0 with probability 1, so we condition the rest of the analysis on G. Let us denote $y_{i_j} = (x_j^*, \sigma_j^*)$ for all $j \in [f]$, and let $\mathcal{I}$ be the set of parties corrupted by $\mathcal{A}$.

Let us consider the following events, for $\ell = 1, \ldots, M$, let us define the following bad events:

1. $\mathsf{E}_{\delta,\ell} : \delta_\ell = 0$ for $\ell = 1, \ldots, M$
2. $\mathsf{E}_{x,j,\ell,1} : x_{\ell,j} = x'_\ell$ for some $j \in \{f+1, \ldots, t\}$ and $\ell \in [M]$
3. $\mathsf{E}_{x,j,\ell,\mathcal{I}} : x_{\ell,j} = x_k^*$ for some $k \in [f]$
4. $\mathsf{E}_{x,j,\ell,2} : x_{\ell,j} = x_{\ell,i}$ for some $i \in \{f+1, \ldots, j-1\}$
5. $\mathsf{E}_\ell : x'_\ell = x'_j$ for some $j < \ell$
6. $\mathsf{E}_{\ell,\mathcal{I}} : x'_\ell = x_j^*$ for some $j \in [f]$
7. $\mathsf{E}_{\varphi,\ell} : h_\varphi(x'_\ell) = 0$ for any $\ell \in [M]$ and fixed public value φ.
8. $\mathsf{E}_{1,\ell} : \mathcal{R}(\varphi, y_{\ell,f+1}, \ldots, y_{\ell,t}, y'_\ell) = \mathsf{Recon}(\varphi, y_{i_1}, \ldots, y_{i_f}, y_{\ell,f+1}, \ldots, y_{\ell,t}, y'_\ell)$, i.e, $\mathcal{R}$ outputs correct s_ℓ for y'_ℓ
9. $\mathsf{E}_{2,\ell} : \mathcal{R}(\varphi, y_{\ell,f+1}, \ldots, y_{\ell,t}, y''_\ell) = \mathsf{Recon}(\varphi, y_{i_1}, \ldots, y_{i_f}, y_{\ell,f+1}, \ldots, y_{\ell,t}, y''_\ell)$, i.e, $\mathcal{R}$ outputs correct s_ℓ for y''_ℓ

We denote $\mathsf{E}_{x,j,\ell} := \mathsf{E}_{x,j,\ell,1} \vee \mathsf{E}_{x,j,\ell,\mathcal{I}} \vee \mathsf{E}_{x,j,\ell,2}$ for all $j \in \{f+1, \ldots, t\}$ and $\ell \in [M]$. Let $\mathcal{J}$ denote the set $\{x_j^*\}_{j\in[f]}$, let $\mathcal{J}_\ell = \{x_{\ell,j}\}_{j\in\{f+1,\ldots,t\}}$ and let $\mathcal{J}_\ell^* = \varphi \cup \mathcal{J}^* \cup \mathcal{J}_\ell \cup \{x'_\ell\}$. Thus, in the event $\neg\mathsf{E}_{\delta,\ell} \wedge_{j\in\{f+1,\ldots,t\}} (\neg\mathsf{E}_{x,j,\ell}) \wedge \neg\mathsf{E}_\ell \wedge \neg\mathsf{E}_{\ell,\mathcal{I}} \wedge \neg\mathsf{E}_{\varphi,\ell} \wedge \mathsf{E}_{1,\ell} \wedge \mathsf{E}_{2,\ell}$, we have that $|\mathcal{J}_\ell^*| = t + 1$ and

$$s_\ell = \sum_{x_{-j}\in\varphi}\left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x_{-j}\}}\frac{x}{x-x_{-j}}\right)\sigma_{-j} + \sum_{x^*_j\in\mathcal{J}^*}\left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x^*_j\}}\frac{x}{x-x^*_j}\right)\sigma^*_j + \sum_{x_{\ell,j}\in\mathcal{J}_\ell}\left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x_{\ell,j}\}}\frac{x}{x-x_{\ell,j}}\right)\sigma_{\ell,j} + \left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x'_\ell\}}\frac{x}{x-x'_\ell}\right)\sigma'_\ell$$

and

$$s'_\ell = \sum_{x_{-j}\in\varphi}\left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x_{-j}\}}\frac{x}{x-x_{-j}}\right)\sigma_{-j} + \sum_{x^*_j\in\mathcal{J}^*}\left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x^*_j\}}\frac{x}{x-x^*_j}\right)\sigma^*_j + \sum_{x_{\ell,j}\in\mathcal{J}_\ell}\left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x_{\ell,j}\}}\frac{x}{x-x_{\ell,j}}\right)\sigma_{\ell,j} + \left(\prod_{x\in\mathcal{J}^*_\ell\setminus\{x'_\ell\}}\frac{x}{x-x'_\ell}\right)(\sigma'_\ell+\delta_\ell)$$

Thus we yield,

$$s'_\ell - s_\ell = \delta_\ell\cdot\left(\prod_{x_{\ell,j}\in\mathcal{J}_\ell}\frac{x_{\ell,j}}{x_{\ell,j}-x'_\ell}\right)\cdot\left(\prod_{x^*_j\in\mathcal{J}^*}\frac{x^*_j}{x^*_j-x'_\ell}\right)\cdot\left(\prod_{x_j\in\varphi}\frac{x_j}{x_j-x'_\ell}\right)$$

Let us define a polynomial $g^*(x)=\prod_{x^*_j\in\mathcal{J}^*}\frac{x^*_j-X}{x^*_j}$. Then, the algorithm Trace returns a correct evaluation z_ℓ of $g^*(X)$ at $X=x'_\ell$ in ℓ-th iteration. Thus, for each $\ell\in[M]$ we obtain a correct evaluation of $g^*(X)$ at a unique point x'_ℓ in the event $\neg\mathsf{E}_{\delta,\ell}\wedge_{j\in\{f+1,\ldots,t\}}(\neg\mathsf{E}_{x,j,\ell})\wedge\neg\mathsf{E}_\ell\wedge\neg\mathsf{E}_{\ell,\mathcal{I}}\wedge\neg\mathsf{E}_{\varphi,\ell}\wedge\mathsf{E}_{1,\ell}\wedge\mathsf{E}_{2,\ell}$. We bound the probability of this event.

Probability Bound

1. $\Pr[\mathsf{E}_{\delta,\ell}]=\frac{1}{p}$ for all $\ell\in[M]$ as δ_ℓ is uniformly sampled from $\mathbb{F}$ for each ℓ.
2. $\Pr[\mathsf{E}_{x,j,\ell,1}]=\frac{1}{p}$, $\mathsf{Pr}[\mathsf{E}_{x,j,\ell,\mathcal{I}}]\leq\frac{f}{p}$, $\mathsf{Pr}[\mathsf{E}_{x,j,\ell,2}]\leq\frac{t-f}{p}$ for all $\ell\in[M]$ and all $j\in\{f+1,\ldots,t\}$. Hence, by union over j we obtain $\Pr[\mathsf{E}_{x,j,\ell}]\leq\frac{t+1}{p}$.
3. $\Pr[\mathsf{E}_\ell\vee\mathsf{E}_{\ell,\mathcal{I}}]\leq\frac{N+f}{p}$ for all $\ell\in[M]$.
4. $\Pr[\mathsf{E}_{\varphi,\ell}]\leq\frac{n-t-1}{p}$ for each $\ell\in[M]$ by Schwarz-Zippel Lemma.
5. Since $\mathcal{R}$ is $(n,t,\boldsymbol{y},\epsilon)$-good we have $\Pr[\mathsf{E}_{1,\ell}]$ and $\Pr[\mathsf{E}_{2,\ell}]$ are both at least ϵ. Let W_ℓ be the random variabls $(\{(x_{\ell,j},\sigma_{\ell,j})\}_{j\in[f+1,t]},x'_\ell)$. Then by the total law of probability and independence of $\mathsf{E}_{1,\ell}|W_\ell$ and $\mathsf{E}_{2,\ell}|W_\ell$ and Jensen's inequality we obtain that $\Pr[\mathsf{E}_{1,\ell}\wedge\mathsf{E}_{2,\ell}]\geq\epsilon^2$.

Thus we obtain the following,

$$\begin{aligned}
&\Pr\left[\neg\mathsf{E}_{\delta,\ell} \bigwedge_{j\in\{f+1,\ldots,t\}} (\neg\mathsf{E}_{x,j,\ell}) \wedge \neg\mathsf{E}_{\ell} \wedge \neg\mathsf{E}_{\ell,\mathcal{I}} \wedge \neg\mathsf{E}_{\varphi,\ell} \wedge \mathsf{E}_{1,\ell} \wedge \mathsf{E}_{2,\ell}\right] \\
&\geq \Pr[\mathsf{E}_{1,\ell} \wedge \mathsf{E}_{2,\ell}] - \Pr\left[\mathsf{E}_{\ell} \vee \mathsf{E}_{\delta,\ell} \vee \mathsf{E}_{\ell,\mathcal{I}} \vee \mathsf{E}_{\varphi,\ell} \bigvee_{j\in[t]\setminus[f]} \mathsf{E}_{x,j,\ell}\right] \\
&\geq \epsilon^2 - 1/p - (M+f)/p - (t-f)(t+1)/p - (n-t-1)/p \\
&\geq \epsilon^2 - (M+t^2+n)/p \\
&\geq \epsilon^2/2
\end{aligned}$$

The last inequality follows from the fact $\epsilon^2 \geq 2(M+n+t^2)/p$. Let us define an indicator random variable $Z_\ell = \mathbb{I}[\neg\mathsf{E}_{\delta,\ell} \wedge_{j\in\{f+1,\ldots,t\}} (\neg\mathsf{E}_{x,j,\ell}) \wedge \neg\mathsf{E}_{\ell} \wedge \neg\mathsf{E}_{\ell,\mathcal{I}} \wedge \neg\mathsf{E}_{\varphi,\ell} \wedge \mathsf{E}_{1,\ell} \wedge \mathsf{E}_{2,\ell}]$, clearly $Z_\ell = 1$ iff we get a correct evaluation of $g^*(X)$ at a unique x'_ℓ. Since all the shares are sampled independently for each $\ell \in [M]$ we obtain $\Pr[Z_\ell = 1] = \mathbb{E}[Z_\ell] \geq \epsilon^2/2$.

Let $Z = \sum_{\ell\in[M]} Z_\ell$. By the Chernoff bound, we have that for every $\eta > 0$,

$$\Pr[Z \leq (1-\eta)\epsilon^2 M/2] \leq e^{-\frac{\epsilon^2 M \eta^2}{4}}$$

Since $N = (2rC)/\epsilon^2$, we can set $\eta = 1 - 1/r$ and thus we obtain,

$$\Pr[Z > C] \geq 1 - e^{-\frac{\epsilon^2 M(1-1/r)^2}{4}}$$

Thus with the probability $1 - e^{-\frac{\epsilon^2 M(1-1/r)^2}{4}}$ the algorithm $\mathcal{D}_{\mathsf{GS}}$ will output the polynomials with at least C evaluations out of $\{x'_\ell, z_\ell\}_{\ell\in[M]}$, including the polynomial $g^*(X)$.

Also, we claim that with high probability there will be no other polynomial in the list G after Step 4. Let us define an event $\mathsf{E}_{g,i}$ for all honest parties $i \in [n-1] \setminus \mathcal{I}$, denoting that x_i is a root of some polynomial in the list G. We observe that shares of the honest parties, i.e., $\{x_i, \sigma_i\}_{i\notin\mathcal{I}}$ are statistically independent from both the view of $\mathcal{A}$ and the shares sampled by the Trace algorithm. Hence, the probability that x_i is a root of any polynomial g_j in G is bounded by f/p by Schwarz-Zippel Lemma. Since G contains τ polynomials we obtain that $\Pr[E_{g,i}] \leq f \cdot \tau/p$. Then, by union bound over all honest parties we obtain

$$\Pr\left[\bigvee_{i\in[n-1]\setminus\mathcal{I}} \mathsf{E}_i\right] \leq \frac{f\cdot(n-1-f)\cdot\tau}{p}$$

Finally, observe that $\mathcal{A}$ can win the game only if (a) $Z < C$ so that $\mathcal{D}_{\mathsf{GS}}$ falis to return g^* or (b) if either events $\mathsf{E}_{g,i}$ occur, causing an honest party to be blamed. Thus, we obtain the advantage,

$$\mathsf{Adv}^{\mathsf{trace}}_{\mathcal{A},\mathsf{TTSS},\varepsilon}(\lambda) \leq e^{\frac{-\epsilon^2 M}{2}\cdot(1-1/r)^2} + \frac{f\cdot(n-f-1)\cdot\tau}{p}$$

C Non-Imputability of TBUSS

Let us recall the Non-Imputability game defined in section 2.3. To achieve non-imputability in Traceable Bottom-Up Secret Sharing (TBUSS), we enhance the scheme to prevent malicious tracers from falsely accusing honest parties. The original construction (Definition 3 and Section 3.1) exposes the random field elements x_j in the tracing key tk and verification key vk, allowing adversaries to trivially impute any honest party by including their x_j in a forged proof. To address this, we use a one-way function $\mathcal{F} : \mathbb{F} \to \mathbb{F}$. Intuitively, we "hide" each party's random evaluation point x_j behind a one-way function, yet still allow any extracted root to be linked back to its owner. The enhanced scheme, Non-Imputable TBUSS (NITBUSS), modifies the algorithms as follows.

C.1 Modified Algorithms for NITBUSS

In the following, we describe the Share, Trace and Verify algorithms and skip the Recon algorithm as it is easy to write following the Share algorithm.

1. $\mathsf{Share}(s, \boldsymbol{\sigma}_B, B)$
   - Define a polynomial q of degree $n-1$ over $\mathbb{F}$ such that
     - $q(0) = s$
     - Parse $\boldsymbol{\sigma}_B \leftarrow \{(x_j, \sigma_j)\}_{j\in B}$
     - Sample $x_{-1}, x_{-2}, \ldots, x_{-(n-t-1)} \xleftarrow{\$} \mathbb{F} \setminus \{x_j\}_{j\in B}$.
     - For all $j \in B$ and set $q(x_j) := \sigma_j$
   - Compute $\varphi := (\varphi_1, \varphi_2, \ldots, \varphi_{n-t-1})$, where $\varphi_i = (x_{-i}, q(x_{-i}))$.
   - For each $j \in [n-1]$ (corresponding parties in B):
     - Compute $u_j = \mathcal{F}(x_j)$
   - Set $\mathsf{tk} = (u_1, u_2, \ldots, u_{n-1})$ and $\mathsf{vk} = (u_1, u_2, \ldots, u_{n-1})$.
   - Output $(\varphi, \mathsf{tk}, \mathsf{vk})$
2. $\mathbf{Trace}^{\mathcal{R}}_{\mathsf{NITBUSS}}(\mathsf{tk}, 1^{1/\epsilon})$
   - Parse tk as $(u_1, u_2, \ldots u_n)$.
   - Run the Trace algorithm to obtain a list of candidate polynomials $G = \{g_j\}_j$ from list decoding. For each candidate polynomial g_j, do:
     - Factor g_j to extract roots $w_1, \ldots, w_f$.
     - For each root w_i:
       - Compute $u'_i = \mathcal{F}(w_i)$
       - If $u'_i \notin \{u_1, \ldots, u_f\}$, discard g_j.
   - If candidates remain, take the first valid $g^* \in G$ with roots $w^*_1, \ldots, w^*_f$.

- For each w_i^*, find the index $k \in [n-1]$ such that $\mathcal{F}(w_i^*) = u_k$ and add k to $\mathcal{I}$.
- Set $\pi \leftarrow (w_1^*, \ldots, w_f^*)$.
- Output $(\mathcal{I}, \pi)$.

3. **Verify**$(\mathsf{vk}, \mathcal{I}, \pi)$
   - Parse $\mathsf{vk} = (u_1, u_2, \ldots, u_{n-1})$, $\mathcal{I} = \{i_1, i_2, \ldots, i_f\} \subset [n-1]$, and $\pi = (w_1, \ldots, w_f)$
   - For each $k \in [f]$:
     - Check $\mathcal{F}(w_k) = u_{i_k}$
   - Output 1 if all checks pass; else, output 0.

The following theorem formally establishes the advantage of non-imputability of NITBUSS. We omit the proof and include it in the full version of this paper.

Theorem 6 (Non-Imputability of NITBUSS). *Let $\mathcal{F}$ be a one-way function. For every PPT adversary $\mathcal{A}$ winning the non-imputability game for* NITBUSS*, there exists a PPT algorithm (inverter) $\mathcal{B}$ such that for every $\lambda \in \mathbb{N}$ it holds that,*

$$\mathsf{Adv}^{\mathsf{ni}}_{\mathcal{A},\mathsf{NITBUSS}}(\lambda) = \Pr[\mathcal{F}(\mathcal{B}(\mathcal{F}(x^*))) = \mathcal{F}(x^*)]$$

where $x \xleftarrow{\$} \mathbb{F}$ and the probability is also over the random coins of $\mathcal{B}$.

C.2 Non-Imputability of NITBUSS

Theorem 7 (Non-Imputability of NITBUSS). *Let $\mathcal{F}$ be a one-way function. For every PPT adversary $\mathcal{A}$ winning the non-imputability game for* NITBUSS*, there exists a PPT algorithm (inverter) $\mathcal{B}$ such that for every $\lambda \in \mathbb{N}$ it holds that,*

$$\mathsf{Adv}^{\mathsf{ni}}_{\mathcal{A},\mathsf{NITBUSS}}(\lambda) = \Pr[\mathcal{F}(\mathcal{B}(\mathcal{F}(x^*))) = \mathcal{F}(x^*)]$$

where $x \xleftarrow{\$} \mathbb{F}$ and the probability is also over the random coins of $\mathcal{B}$.

Proof. We write an overview of the proof. Construct $\mathcal{B}$ as follows:

1. $\mathcal{B}$ receives a challenge $u^* = \mathcal{F}(x^*)$ for $x^* \xleftarrow{\$} \mathbb{F}$.
2. $\mathcal{A}$ outputs $i^* \in [n-1]$ (A party to frame)
3. For all $j \in [n-1] \setminus \{i^*\}$:
   - Sample $x_j \xleftarrow{\$} \mathbb{F}$ and compute $u_j = \mathcal{F}(x_j)$.
4. For party i^* set $\mathsf{tk}_{i^*} = \mathsf{vk}_{i^*} = u^*$.
5. Generate $\mathsf{tk} = \mathsf{vk} = (u_1, u_2, \ldots, u_{n-1})$.
6. To generate secret shares for every $j \in [n-1] \setminus \{i^*\}$, $\mathcal{B}$ computes $\sigma_j = f(x_j)$ and sets $y_j = (x_j, \sigma_j)$.
7. Give $(\mathsf{tk}, \mathsf{vk}, \{y_j\}_{j \in [n] \setminus \{i^*\}})$ to $\mathcal{A}$.
8. $\mathcal{A}$ outputs $\mathcal{J}^*$ and $\pi = (w_1, w_2, \ldots, w_f)$.

9. If $i^* \in \mathcal{J}^*$ and $\mathsf{Verify}(\mathsf{vk}, \mathcal{J}^*, \pi) = 1$ then for the w_k corresponding to i^*: $\mathcal{F}(w_k) = u_{i^*} = u^*$.
10. $\mathcal{B}$ outputs w_k as a pre-image of u^*.

$\mathcal{B}$ perfectly simulates $\mathcal{A}$'s view: the keys and the shares are identically distributed to the real scheme. If $\mathcal{A}$ wins, Verify ensures that $\mathcal{F}(w_k) = u_{i^*} = u_*$, so $\mathcal{B}$ successfully inverts $\mathcal{F}$ at u^*. Hence, the theorem holds true. □

D Non-Imputability of TSKR

Recall that $\mathsf{TSKR} = (\Pi_{\mathsf{Init}}, \Pi_{\mathsf{Back}}, \Pi_{\mathsf{Rec}})$ where Π_{Back} and Π_{Rec} invoke $\mathsf{TBUSS.Share}$ and $\mathsf{TBUSS.Recon}$, respectively. To address the risk of malicious tracers falsely accusing honest parties in TSKR (Section 6.1), we introduce Non-Imputable Social Key Recovery (NISKR). This scheme uses the Non-Imputable TBUSS ($\mathsf{NITBUSS}$) construction from Section C, which uses a one-way function $\mathcal{F} : \mathbb{F} \to \{0,1\}^*$ to hide the random evaluation points $x_{i,j}$ of guardians. By replacing TBUSS with $\mathsf{NITBUSS}$ in the SKR protocols, we ensure that, the tracer receives $\mathcal{F}(x_{i,j})$ instead of $x_{i,j}$, preventing trivial forgery of proofs. Valid proofs must link roots of traced polynomials to preimages of $\mathcal{F}(x_{i,j})$, making false accusations computationally infeasible by the hardness of inverting $\mathcal{F}$. We define the modified Algorithms for $\mathsf{NITBUSS}$ below as

- Define NISKR as the tuple $(\Pi_{\mathsf{Init}}, \Pi_{\mathsf{Back}}, \Pi_{\mathsf{Rec}})$ where Π_{Back} and Π_{Rec} invoke $\mathsf{NITBUSS}$ scheme:
  - Π_{Back} uses $\mathsf{NITBUSS.Share}$ to generate backup strings.
  - Π_{Rec} uses $\mathsf{NITBUSS.Recon}$ to recover secrets.

Thus the full-fledged NISKR scheme will be the following:

Non-Imputable Social Key Recovery (NISKR)

$\Pi_{\mathsf{Init}}(1^\lambda, 1^N)$: Each party P_i runs the key generation algorithm $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and publishes their public key pk_i and stores their secret key sk_i.

$\Pi_{\mathsf{Back}}(i, B, \mathbf{pk}_{\boldsymbol{B}}, \mathbf{sk}_{R\cup\{i\}})$: Party P_i runs this as a key-owner with a set of $(n-1)$ guardians $\{P_j\}_{j\in B}$ as follows:
- For each guardian P_j, P_i sends a backup request to P_j.
- Each P_j computes $x_{i,j} := H(j, \mathsf{sk}_j, \mathsf{pk}_i)$ and $\sigma_{i,j} = H(i, \mathsf{sk}_j)$ and sends $(x_{i,j}, \sigma_{i,j})$ back to the party P_i.
- On receiving $(x_{i,j}, \sigma_{i,j})$, and P_i defines $s := \mathsf{sk}_i$ computes: $\mathsf{NITBUSS.Share}(\mathsf{sk}_i, \{(x_{i,j}, \sigma_{i,j})\}_{j\in B}, B)$ and output $(\varphi, \mathsf{tk}, \mathsf{vk})$ and publishes $pub := \varphi$.

$\Pi_{\mathsf{Rec}}(i, R, \mathbf{pk}_{[N]}, \varphi, \mathbf{sk}_R)$: The party P_i wants to compute his secret with a set of $t+1$ guardians as follows:
- For each guardian P_j, P_i send a request to P_j for $j \in R$.
- After receiving $(x_{i,j}, \sigma_{i,j})_{j\in R}$, P_i retrieve φ and compute $\mathsf{NITBUSS.Recon}(\varphi, \{(x_{i,j}, \sigma_{i,j})\}_{j\in R}, R)$ to output s'.

$\mathsf{GNon\text{-}Imputability}_{\mathcal{A},\mathsf{TSKR}}(\lambda)$**: The non-imputability game for traceable threshold Social Key Recovery** TSKR**.**

1. $\mathcal{A}(1^\lambda, n, t)$ outputs $(i^*, \{s_k\}_{k\in[N]}, \{B_k\}_{k\in[N]}, \mathsf{state})$.
2. Run: $\Pi_{\mathsf{Back}}(s_k, B_k, \boldsymbol{\sigma}_{B_k})$ outputs $(\varphi_k, \mathsf{tk}^{(k)}, \mathsf{vk}^{(k)})$.
3. On input all shares *except* for the i^*-th one and the keys tk and vk, $\mathcal{A}(\mathsf{state}, \{\boldsymbol{\sigma}_{i,j}\}_{i,j\in[N]} \setminus \{\sigma_{i,i*}\}_{i\in[N]}, \mathsf{tk}, \mathsf{vk})$ outputs $(\mathcal{I}^*, \pi)$.
4. $\mathcal{A}$ wins if $i^* \in \mathcal{I}^*$ and $\mathsf{Verify}(\mathsf{vk}, \mathcal{I}^*, \pi) = 1$.

Fig. 7: The Tracing Algorithm for the Traceable Social Key Recovery TSKR.

– Then the party P_i checks the validity of the key pair (s', pk_i). If yes then privately output $\mathsf{sk}_i = s'$, else output $\perp$.

See the Non-Imputability game for SKR in Figure 7.

Theorem 8 (Non-Imputability of NISKR**).** *Let $\mathcal{F}$ be a one-way function. For every PPT adversary $\mathcal{A}$ winning the non-imputability game for* NISKR*, there exists a PPT algorithm (inverter) $\mathcal{B}$ such that for every $\lambda, N \in \mathbb{N}$ it holds that,*

$$\mathsf{Adv}^{\mathsf{ni}}_{\mathcal{A},\mathsf{NISKR}}(\lambda, N) = \Pr[\mathcal{F}(\mathcal{B}(\mathcal{F}(x^*))) = \mathcal{F}(x^*)]$$

where $x \xleftarrow{\$} \mathbb{F}$ and the probability is also over the randomness $\mathcal{B}$.

The proof of this theorem is exactly similar like the proof of the Theorem 7, hence omitted. NISKR enhances TSKR with non-imputability by integrating NITBUSS. This ensures accountability while preventing false accusations, with security resting on standard cryptographic assumptions.